

Pontifícia Universidade Católica do Rio Grande do Sul
Programa de Pós-Graduação em [nome do programa — preencher]

Marcelo Claro Laranjeira

**ROTEAMENTO INSPIRADO EM ATENÇÃO: FORMALIZAÇÃO DO NÚCLEO
MATEMÁTICO EM LEAN 4 E AVALIAÇÃO EMPÍRICA INTERNA**

Porto Alegre
2026

Marcelo Claro Laranjeira

**Roteamento inspirado em atenção: formalização do núcleo matemático em Lean 4 e
avaliação empírica interna**

Dissertação apresentada ao Programa de Pós-Graduação em [nome do programa — preencher] da Pontifícia Universidade Católica do Rio Grande do Sul como requisito parcial para obtenção do título de Mestre em [denominação do título — preencher]. Área de concentração: [preencher]. Linha de pesquisa: [preencher].

Orientação: [nome e titulação do orientador — preencher].

Porto Alegre
2026

Ficha catalográfica

[Inserir a ficha emitida pelo serviço de catalogação da instituição após a confirmação dos dados autorais e da versão final.]

Marcelo Claro Laranjeira

**Roteamento inspirado em atenção: formalização do núcleo matemático em Lean 4 e
avaliação empírica interna**

Dissertação apresentada ao Programa de Pós-Graduação em [nome do programa — preencher] da Pontifícia Universidade Católica do Rio Grande do Sul como requisito parcial para obtenção do título de Mestre em [denominação do título — preencher]. Área de concentração: [preencher]. Linha de pesquisa: [preencher].

Data da aprovação: [preencher somente após a defesa].

Banca examinadora: [nomes, titulações, instituições e assinaturas — preencher].

[A aprovação e as assinaturas ainda não foram registradas nesta versão.]

AGRADECIMENTOS

Às comunidades Lean e Mathlib, pela manutenção das ferramentas e bibliotecas utilizadas, e às iniciativas de ciência aberta que apoiam a documentação e a consulta das fontes deste trabalho.

RESUMO

Este trabalho investiga a formalização do núcleo matemático e a avaliação empírica interna de um mecanismo de roteamento de tarefas entre agentes de software, inspirado na ponderação por atenção. A pesquisa combina provas em Lean 4 com Mathlib, inspeção dos artefatos de execução, uma bancada sintética e testes aleatórios de propriedades numéricas. Os registros locais documentam onze lemas sobre normalização do softmax, limites de combinações convexas, existência de maximizador finito, invariância a deslocamento, pesos positivos, caso de candidato único, monotonicidade, ordenação top- k , temperatura e limite superior da exponencial após deslocamento. As provas referem-se à aritmética real exata e não certificam integralmente a implementação em ponto flutuante. Na bancada principal, com 200 decisões distribuídas por cinco sementes, o arrependimento médio registrado foi 0,042, com margem exploratória de 0,011, frente a valores entre 0,19 e 0,22 dos comparadores simples. Ablações em um conjunto distinto de instâncias indicam dependência da confiança, construída como proxy da qualidade latente. A variação do parâmetro do oráculo apenas reescala a perda entre candidatos plenamente elegíveis. Os intervalos desconsideram o agrupamento por piscina de agentes e devem ser interpretados como resumos exploratórios. O relatório numérico registra 20 000 vetores sem violações acima das tolerâncias examinadas. Conclui-se que os artefatos sustentam propriedades do modelo abstrato e resultados preliminares no simulador, permanecendo pendentes a certificação numérica completa, a análise estatística com agrupamento, a comparação externa e a reprodução independente.

Palavras-chave: roteamento de agentes; atenção; Lean 4; verificação formal; avaliação empírica; reprodutibilidade.

ABSTRACT

This dissertation investigates the formalization of the mathematical core and the internal empirical evaluation of a task router for software agents inspired by attention-based weighting. The study combines proofs in Lean 4 with Mathlib, inspection of execution artifacts, a synthetic benchmark, and randomized tests of numerical properties. Local records document eleven lemmas on softmax normalization, convex-combination bounds, existence of a finite maximizer, shift invariance, positive weights, the singleton case, monotonicity, top- k ordering, temperature, and an upper bound on shifted exponentials. The proofs concern exact real arithmetic and do not fully certify the floating-point implementation. In the main benchmark, comprising 200 decisions across five seeds, recorded mean regret was 0.042, with an exploratory margin of 0.011, compared with values between 0.19 and 0.22 for simple baselines. Ablations on a distinct set of instances indicate dependence on confidence, which was constructed as a proxy for latent quality. Changing the oracle parameter only rescales regret among fully eligible candidates. The reported intervals disregard clustering within agent pools and should be interpreted as exploratory summaries. The numerical report records 20,000 vectors with no violations above the tested tolerances. The artifacts support properties of the abstract model and preliminary findings within the simulator. Full numerical certification, statistical analysis accounting for clustering, external comparison, and independent reproduction remain pending.

Keywords: agent routing; attention; Lean 4; formal verification; empirical evaluation; reproducibility.

LISTA DE TABELAS

Tabela 1	– Concentração por temperatura em $[1, 2, 3]$ (calculado pela implementação).	16
Tabela 2	– Analogia Transformer $\leftrightarrow$ roteador (sem transferência de propriedades).	19
Tabela 3	– Táticas usadas nas onze provas (guia de leitura).	22
Tabela 4	– Auditoria de axiomas (<code>#print axioms</code>).	28
Tabela 5	– Ablações: arrependimento por parâmetro do oráculo e diferença em relação à atenção (200 decisões por configuração).	31
Tabela 6	– Diferenças pareadas por decisão vs. atenção ($q_w = 0,6$) e taxa de vitória/empate.	32
Tabela 7	– Arrependimento por estratégia nos três valores do parâmetro do oráculo (200 decisões cada).	32
Tabela 8	– Arrependimento por semente e estratégia (40 decisões cada; acerto da atenção entre parênteses).	32
Tabela 9	– Limitações identificadas e requisitos de avaliação futura.	36
Tabela 10	– Programa de trabalhos futuros (pacotes com aceite).	37
Tabela 11	– Pinos verificados por <code>git ls-remote</code> (2026-10-09) e SHA-256 das licenças.	50
Tabela 12	– Ciclos de evolução da linhagem Lean-roteamento (registros históricos).	88

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
IC	Intervalo de confiança
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
MCP	Model Context Protocol
OSF	Open Science Framework
PUCRS	Pontifícia Universidade Católica do Rio Grande do Sul
Q/K/V	Consulta, chave e valor (<i>query, key, value</i>)
SDD	Desenvolvimento orientado a especificação
SMT	Satisfatibilidade módulo teorias
TDD	Desenvolvimento orientado a testes

LISTA DE SÍMBOLOS

w_i	Peso de atenção do candidato i
f_i	Escore do candidato i
τ	Temperatura positiva do softmax
q	Qualidade latente do agente no simulador
q_w	Peso da qualidade no oráculo sintético
$u(a)$	Utilidade combinada do candidato a
α_h	Peso fixo da cabeça h

SUMÁRIO

1	INTRODUÇÃO	12
1.1	O problema	12
1.2	Questão e objetivos	12
1.3	Justificativa	12
1.4	Delimitação e tese defendida	13
1.5	Estrutura do trabalho	13
1.6	Cenário-guia de roteamento (saídas registradas)	13
2	PRELIMINARES MATEMÁTICOS E COMPUTACIONAIS	14
2.1	Informação, entropia e exponenciação	14
2.2	Conjuntos convexos e ordens finitas	14
2.3	Atenção neural e tradução automática	14
2.4	Verificação, SMT e inspeção	14
2.5	Ponto flutuante e computação científica	15
2.6	Método empírico e estatística	15
2.7	Exemplos de cálculo	15
2.8	Top-k e complexidade	16
2.9	Cálculo da ponderação exponencial	16
2.10	Ordenação e top-k	16
2.11	Derivação variacional (MaxEnt)	16
2.12	Limites de temperatura	17
2.13	Anatomia do ponto flutuante	17
2.14	Intervalos, Pearson e pareamento	17
2.15	Táticas Lean em prosa	17
2.16	Orquestração: do contrato ao quadro-negro	17
3	FUNDAMENTAÇÃO TEÓRICA	19
3.1	Ponderação exponencial normalizada	19
3.2	Correspondência Q/K/V (parcial e estrutural)	19
3.3	Combinação convexa e seleção gulosa	20
3.4	Assistentes de prova e orquestração multiagente	20
3.5	O mecanismo de atenção original	20
3.6	Temperatura e calibração	20
3.7	Ponto flutuante e certificação	21
3.8	Verificação formal para leigos	21
3.9	Ler uma prova deste trabalho	21

3.10	Táticas de uso frequente	21
3.11	Trabalhos relacionados	22
3.12	Fichamento por obra (o que cada uma informa)	23
4	MÉTODO	24
4.1	Provas formais	24
4.2	Bancada sintética e conjuntos de instâncias	24
4.3	Parâmetro do oráculo	24
4.4	Testes numéricos e estatística exploratória	25
4.5	Ameaças à validade e considerações éticas	25
4.6	Protocolo de desenvolvimento e reprodução	25
4.7	Interpretação das métricas	25
4.8	Intervalos registrados e seus limites	26
4.9	Pareamento e calibração probabilística	26
5	PROVAS FORMAIS	27
5.1	Demonstrações integrais, teorema por teorema	28
5.1.1	T1 — soma unitária	28
5.1.2	T2/T6 — limitação bilateral convexa	28
5.1.3	T3 — maximizador finito	29
5.1.4	T4 — invariância a deslocamento	29
5.1.5	T5 — pertinência a $(0, 1)$	29
5.1.6	T7 — peso unitário no singleton	29
5.1.7	T8 — monotonicidade estrita	29
5.1.8	T9 — dominância top- k	29
5.1.9	T10 — nitidez por temperatura	29
5.1.10	T11 — limite superior da exponencial após deslocamento	30
5.2	Necessidade das hipóteses (contraexemplos)	30
6	AValiação empírica e testes numéricos	31
6.1	Fuzz em ponto flutuante	31
6.2	Bancada principal e variação do parâmetro do oráculo	31
6.3	Estudo de caso completo: Ana contra Cid	33
6.4	Correlações de Pearson (200 decisões)	33
6.5	Demonstração de roteamento sobre cartões do catálogo	33
6.6	Interpretação das estratégias	33
7	DISCUSSÃO	35
7.1	Alcance das garantias formais	35
7.2	Dependência do desenho sintético	35

7.3	Riscos residuais e avaliação interna	35
7.4	Relação com a literatura e perspectivas	36
8	CONCLUSÃO	37
	Referências	38
	GLOSSÁRIO	44
	APÊNDICE A – CÓDIGO LEAN INTEGRAL	46
	APÊNDICE B – TRILHA DE REPRODUÇÃO	50
	APÊNDICE C – CÓDIGO DOS EXPERIMENTOS E DO PIPELINE	51
	APÊNDICE D – RESULTADOS EM JSON	76
	APÊNDICE E – TRANSCRIÇÃO DA SUÍTE DE TESTES	83
	APÊNDICE F – HISTÓRICO DE AUDITORIA	88
	APÊNDICE G – ESPECIFICAÇÕES FORMAIS	89

1 INTRODUÇÃO

1.1 O problema

Sistemas que delegam tarefas a agentes de software decidem sob incerteza com critérios múltiplos, e essas decisões raramente carregam garantias auditáveis: não se sabe demonstrar que os pesos somam um, que a regra escolhe o melhor pelos critérios declarados, nem que a medição de qualidade significa o que diz. A engenharia de software experimental estabeleceu há décadas que alegações exigem desenho, medição e relato disciplinados (Basili, Selby e Hutchens, 1986; Wohlin et al., 2012; Runeson e Höst, 2009), e que revisões sistemáticas exigem protocolo de relato (Page et al., 2021); a crise de reprodutibilidade documentada em múltiplas áreas (Open Science Collaboration, 2015; Ioannidis, 2005; Collberg e Proebsting, 2016) mostra o custo de evidência fraca. Este trabalho aplica esse padrão a um roteador por atenção: cada propriedade alegada ou é provada por máquina, ou é medida com incerteza declarada, ou é renunciada explicitamente.

1.2 Questão e objetivos

Que propriedades do núcleo matemático de um roteador inspirado em atenção podem ser formalizadas, e que comportamento sua implementação apresenta em uma avaliação sintética interna? Objetivo geral: formalizar e avaliar internamente um roteamento por atenção. Objetivos específicos: (OE1) verificar por máquina o núcleo matemático em Lean 4 com Mathlib (De Moura e Ullrich, 2021; The Mathlib Community, 2020); (OE2) comparar arrependimento e taxa de acerto com os comparadores, relatando intervalos exploratórios, seguindo a recomendação de estimação em vez de significância pontual (Cumming, 2014; Wasserstein e Lazar, 2016); (OE3) auditar axiomas, alegações, referências e código, incluindo inspeção e métricas na tradição de (Fagan, 1976; McCabe, 1976; Chidamber e Kemerer, 1994); (OE4) declarar limites e publicar a trilha de reprodução segundo princípios FAIR (Wilkinson et al., 2016) e documentação de dados e modelos (Geburu et al., 2021; Mitchell et al., 2019).

1.3 Justificativa

Roteamento opaco não é auditável; garantias formais exigem hipóteses explícitas; medições requerem análise de incerteza. Como perspectiva educacional, trilhas auditáveis desse tipo oferecem material didático para o ensino de avaliação e validação — hipótese de trabalho futuro, não resultado desta dissertação.

1.4 Delimitação e tese defendida

Defende-se exclusivamente otimalidade relativa à utilidade definida e evidência preliminar interna. Não se compara com plataformas existentes, não se generaliza além do simulador descrito e não se transfere propriedades de Transformers treinados. A tese metodológica é que verificação por máquina, teste numérico de propriedades, ablação e auditoria de alegações compõem um padrão de evidência auditável mesmo sem validação externa.

1.5 Estrutura do trabalho

A Seção 2 fixa preliminares matemáticos; a Seção 3 fundamenta atenção, correspondência Q/K/V, combinação convexa, seleção gulosa, Lean e os trabalhos relacionados; a Seção 4 detalha método, estatística, ameaças e validade; a Seção 5 apresenta as onze provas comentadas; a Seção 6, os testes de propriedades e a avaliação empírica; a Seção 7 discute revisões e riscos; a Seção 8 conclui; o glossário define os termos principais; os apêndices preservam código, resultados, transcrições e especificações.

1.6 Cenário-guia de roteamento (saídas registradas)

Tarefa “provar teorema formal com código Lean” exigindo formal e código, com três agentes: Ana (prova e código; carga 0,1; confiança 0,9), Bia (código e dados; carga 0,8; confiança 0,5) e Cid (as três capacidades; carga 0,4; confiança 0,7). O gate exclui Bia (falta formal); utilidades: Ana 0,9175, Cid 0,8161; pesos finais Ana 0,5253 e Cid 0,4747 (soma 1). Ana vence com menos capacidades porque combina menor carga, maior confiança e melhor ajuste semântico — exatamente o tipo de compensação que os teoremas disciplinam e a bancada mede.

Dois casos-limite reais: (i) agentes gêmeos que diferem só na descrição recebem 0,5053 contra 0,4947 — nem gêmeos empatam na semântica, e empates exatos decidem-se pelo identificador, deterministicamente; (ii) candidato único recebe peso 1,0 — o T7 em execução.

2 PRELIMINARES MATEMÁTICOS E COMPUTACIONAIS

2.1 Informação, entropia e exponenciação

Os valores numéricos de pesos apresentados no texto são arredondados a quatro casas decimais; a soma unitária refere-se aos valores antes desse arredondamento. A teoria matemática da comunicação (Shannon, 1948) introduziu a entropia como medida de incerteza; a relação entre informação e suficiência foi formalizada por Kullback e Leibler (1951); e a conexão entre teoria da informação e mecânica estatística, com a distribuição exponencial como maximizadora de entropia sob média fixada, por Jaynes (1957). É dessa linhagem que descende a ponderação exponencial normalizada: sob uma restrição adequada de média, a solução tem forma $w_i \propto e^{\beta f_i}$, com β determinado pela restrição; o softmax usual corresponde ao caso particular $\beta = 1$. A interpretação probabilística de saídas de redes feedforward, com relações ao reconhecimento estatístico de padrões, foi estabelecida por Bridle (1990). Regras de escore estritamente próprias, que incentivam probabilidades calibradas, foram caracterizadas por Gneiting e Raftery (2007).

2.2 Conjuntos convexos e ordens finitas

Uma combinação $\sum w_i x_i$ com $w_i \geq 0$ e $\sum w_i = 1$ jamais escapa ao intervalo $[\min x_i, \max x_i]$: é o conteúdo de T2/T6, e a razão formal pela qual pesos devem somar um. Em conjuntos finitos não vazios sempre existe elemento maximal de qualquer função real (T3): finitude elimina a possibilidade de fuga ao infinito que invalida máximos em domínios infinitos. Listas ordenadas de forma decrescente têm a propriedade de que qualquer prefixo domina o sufixo (T9): é a anatomia do top- k .

2.3 Atenção neural e tradução automática

A atenção produto-escalado com múltiplas cabeças foi introduzida para tradução automática (Vaswani et al., 2017), com variantes efetivas exploradas em tradução neural (Luong, Pham e Manning, 2015), pré-treino bidirecional (Devlin et al., 2019), dependências longas (Dai et al., 2019), benchmarks multitarefa (Wang et al., 2018), compreensão de leitura em larga escala (Rajpurkar et al., 2016), bases hierárquicas de imagens (Deng et al., 2009) e ferramental de processamento (Wolf et al., 2020). Este trabalho herda apenas a forma — ponderação normalizada de cabeças com decisão pelo máximo — sem pesos treinados.

2.4 Verificação, SMT e inspeção

Bases axiomáticas para programação (Hoare, 1969) fundam a verificação dedutiva mecanizada em Lean (De Moura e Ullrich, 2021) e Mathlib (The Mathlib Community, 2020); solucionadores

SMT eficientes como o Z3 (De Moura e Bjørner, 2008) e computação simbólica como SymPy (Meurer et al., 2017) compõem o ecossistema de apoio. Inspeções de projeto e código reduzem erros antes da execução (Fagan, 1976); complexidade ciclomática (McCabe, 1976) e suítes de métricas orientadas a objetos (Chidamber e Kemerer, 1994) disciplinam a auditoria de código; processos racionais de projeto documentam decisões (Parnas e Clements, 1986). Alocação de tarefas distribuídas segue desde o protocolo de rede de contratos (Smith, 1980), e integração de conhecimento sob incerteza desde sistemas de quadro-negro (Erman et al., 1980).

2.5 Ponto flutuante e computação científica

A aritmética de ponto flutuante exige atenção a arredondamento e casos especiais (Goldberg, 1991); o manual de referência consolida algoritmos e análise (Muller et al., 2018). A computação científica em Python apoia-se em arranjos (Harris et al., 2020) e algoritmos fundamentais (Virtanen et al., 2020).

2.6 Método empírico e estatística

Experimentação disciplinada em engenharia de software (Basili, Selby e Hutchens, 1986; Wohlin et al., 2012), diretrizes de estudo de caso (Runeson e Höst, 2009) e relato sistemático (Page et al., 2021) estruturam o desenho; a declaração sobre valores- p (Wasserstein e Lazar, 2016), a nova estatística por estimação (Cumming, 2014) e o bootstrap (Efron, 1979) fundamentam o relato por intervalos em vez de significância pontual. Reprodutibilidade como problema de campo (Open Science Collaboration, 2015; Collberg e Proebsting, 2016; Ioannidis, 2005), dados documentados (Geburu et al., 2021), modelos documentados (Mitchell et al., 2019) e dados geridos por FAIR (Wilkinson et al., 2016) justificam a trilha local de reprodução.

2.7 Exemplos de cálculo

Normalização. Com $S = \sum_j e^{f_j} > 0$, cada quociente e^{f_i}/S é positivo e a soma com denominador comum reduz-se a $S/S = 1$. *Deslocamento.* Subtrair c multiplica numerador e denominador por $1/e^c$, que cancela: a decisão independe do nível. *Temperatura.* Para $\tau_1 < \tau_2$ e maximizador único i^* , a razão de pesos cresce porque cada termo concorrente $e^{(f_j - f_{i^*})/\tau}$ decai mais rápido no τ menor — a Tabela 1 mostra $0,5065 \rightarrow 0,9817$. *Limite da exponencial.* Em reais exatos, deslocar pelo máximo torna cada expoente não positivo e cada exponencial pertence a $(0, 1]$. T11 demonstra esse limite; em ponto flutuante, underflow e arredondamento requerem análise própria.

2.8 Top- k e complexidade

Extrair os k maiores por heap custa $O(n \log k)$ contra $O(n \log n)$ da ordenação total; o desempate por identificador torna a decisão determinística. A garantia independe do algoritmo: sob ordem decrescente, o prefixo domina o sufixo.

2.9 Cálculo da ponderação exponencial

A entropia de uma distribuição, na teoria matemática da comunicação (Shannon, 1948), mede incerteza; dada média fixada dos escores, a distribuição de máxima entropia tem forma exponencial (Jaynes, 1957) — é a origem variacional do softmax, cuja interpretação probabilística em classificação foi estabelecida em (Bridle, 1990). Três cálculos sustentam o uso: (i) normalização, $\sum_i w_i = 1$ por construção (T1); (ii) invariância a deslocamento (T4), que autoriza a subtração do máximo; (iii) limite superior da exponencial após deslocamento por um majorante (T11). A Tabela 1 mostra a nitidez por temperatura em escores $[1, 2, 3]$: o peso máximo sobe $0,5065 \rightarrow 0,6652 \rightarrow 0,8668 \rightarrow 0,9817$ ao esfriar — uma ilustração numérica de T10.

Tabela 1 – Concentração por temperatura em $[1, 2, 3]$ (calculado pela implementação).

τ	w_1	w_2	w_3
2,0	0,1863	0,3072	0,5065
1,0	0,0900	0,2447	0,6652
0,5	0,0159	0,1173	0,8668
0,25	0,0003	0,0180	0,9817

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

2.10 Ordenação e top- k

Ordenar n utilidades custa $O(n \log n)$; extrair os k maiores por heap custa $O(n \log k)$. A implementação ordena tudo e fatia o topo; com 40 agentes e latência de 2,17 ms, a seleção parcial segue como melhoria medida pendente. O desempate é determinístico (menor identificador vence), o que torna a seleção determinística quando entradas, ambiente e demais operações permanecem fixos. A garantia matemática independe do algoritmo: sob ordem pairwise decrescente, o prefixo domina o sufixo (T9).

2.11 Derivação variacional (MaxEnt)

Maximizar $H(w) = -\sum w_i \ln w_i$ sob $\sum w_i = 1$ e média fixada, por multiplicadores de Lagrange, produz $w_i \propto e^{\beta f_i}$, com β determinado pela média imposta: a forma exponencial não é arbitrária, é a distribuição de máxima entropia compatível com a média (Jaynes, 1957; Shannon,

1948). Com $\beta = 1$ recupera-se o softmax; $\beta = 1/\tau$ introduz a temperatura. Regras de escore próprias caracterizam quando probabilidades relatadas são honestas (Gneiting e Raftery, 2007).

2.12 Limites de temperatura

Com maximizador único i^* : quando $\tau \rightarrow 0^+$, cada razão $e^{(f_j - f_{i^*})/\tau} \rightarrow 0$ e $w_{i^*} \rightarrow 1$; quando $\tau \rightarrow \infty$, todas as razões tendem a 1 e os pesos equalizam em $1/n$. T10 é a versão finita e verificada dessa intuição: esfriar aumenta estritamente o peso do favorito, sem passar ao limite.

2.13 Anatomia do ponto flutuante

IEEE-754 representa $x \approx m \times 2^e$ com arredondamento ao mais próximo; operações elementares têm erro relativo limitado pela unidade de arredondamento, mas exponenciação de argumentos grandes estoura para infinito (Goldberg, 1991; Muller et al., 2018). Subtrair o máximo antes de exponenciar mantém todo expoente ≤ 0 (T11), logo cada termo fica em $(0, 1]$ em aritmética real exata. Esse limite não certifica as operações IEEE-754: exponenciais pequenas podem sofrer underflow, e a invariância exata de T4 não implica igualdade bit a bit para quaisquer entradas.

2.14 Intervalos, Pearson e pareamento

Para média $\bar{x}$ de n observações com desvio s , o IC95 $\bar{x} \pm 1,96s/\sqrt{n}$ aproxima a incerteza sob normalidade; o relato por intervalos segue a estatística por estimação (Cumming, 2014) e a advertência contra o uso pontual de valores- p (Wasserstein e Lazar, 2016); reamostragem bootstrap (Efron, 1979) é a alternativa quando a normalidade é suspeita. Pearson mede relação linear; parear estratégias nas mesmas decisões controla diferenças de dificuldade entre comparadores. Regras de escore próprias (Gneiting e Raftery, 2007) lembram que probabilidade bem relatada é hipótese, sem presunção de validade.

2.15 Táticas Lean em prosa

Positividade (exp_pos, divisão de positivos) estabelece sinais. *Reescrita* (sum_div, sum_mul, exp_sub, exp_add) move termos entre formas equivalentes. *Monotonicidade* (mul_le_mul_of_nonneg_left, div_lt_div_iff) preserva desigualdades. *Fechamento* (div_self, linarith, field_simp, ring) conclui identidades. *Estrutura* (take_append_drop, pairwise_append, add_sum_erase, exists_max_image, sum_lt_sum, sum_eq_single_of_mem) decompõe objetos finitos. Cada nome foi conferido na Mathlib pinada, nunca na memória.

2.16 Orquestração: do contrato ao quadro-negro

Alocar tarefas entre resolvidores distribuídos remonta ao protocolo de rede de contratos (Smith, 1980); integrar conhecimento heterogêneo sob incerteza, aos sistemas de quadro-negro (Erman

et al., 1980). Nosso roteador instancia a linhagem com meios atuais: gates decidem elegibilidade, cabeças pontuam aspectos, softmax normaliza, máximo decide — cada camada auditável separadamente, como documenta a trilha de pinos e hashes.

3 FUNDAMENTAÇÃO TEÓRICA

Antes de provar ou medir, fixamos o vocabulário: o que a ponderação exponencial garante, onde ela se inspira e onde a analogia termina. A base axiomática da programação (Hoare, 1969) sustenta a verificação dedutiva; a computação numérica e simbólica executa-se sobre arranjos e algoritmos fundamentais (Harris et al., 2020; Virtanen et al., 2020), com apoio simbólico e SMT (Meurer et al., 2017; De Moura e Bjørner, 2008).

3.1 Ponderação exponencial normalizada

A soma unitária dos pesos não demonstra calibração probabilística. A avaliação deste trabalho mede arrependimento e acerto, sem ajuste de temperatura, aprendizagem de pesos ou estimação de métricas de calibração. Dados escores finitos $f_1, \dots, f_n$, os pesos $w_i = e^{f_i} / \sum_j e^{f_j}$ somam 1 (T1), situam-se em $(0, 1)$ com ao menos dois candidatos (T5) e valem 1 no singleton (T7); deslocar escores por constante nada altera (T4), autorizando a subtração do máximo por estabilidade numérica. A calibração da temperatura, amplamente estudada como parâmetro de nitidez e calibração de classificadores (Komisarenko e Kull, 2024; Xuan, Yang e Li, 2025; Xie et al., 2024), aparece aqui em forma elementar, demonstrada por máquina (T10): esfriar aumenta o peso do maximizador único. A verificação de programas com aritmética de ponto flutuante exige análise própria (Bagnara et al., 2019; Murray, 2026); este trabalho documenta a ponte empírica (fuzz) e um lema de limite superior da exponencial após deslocamento (T11), sem reivindicar certificação total de floats.

3.2 Correspondência Q/K/V (parcial e estrutural)

Tabela 2 – Analogia Transformer $\leftrightarrow$ roteador (sem transferência de propriedades).

Transformer	Roteador	Natureza
Consulta (Q)	Vetor da tarefa	Estrutural
Chave (K)	Vetor do cartão do agente	Estrutural
Escore QK	Similaridade + 4 cabeças	Parcial (sem produto escalado/temperatura)
Valor (V)	(utilidade, identificador)	Parcial (sem mistura de valores)
Agregação	Convexa + softmax	Demonstrada (T2, T6)
Decisão	Máximo / top- k	Demonstrada (T3, T9)

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

3.3 Combinação convexa e seleção gulosa

Com pesos não negativos somando 1, a utilidade $u(a) = \sum_h \alpha_h s_h(a)$ situa-se entre mínimo e máximo dos componentes (T2, T6): conteúdo formal do validador que rejeita somas distintas de 1. Em conjunto finito não vazio há maximizador (T3); o padrão top- k domina o resto sob ordem pairwise (T9); o peso cresce com o próprio escore (T8). Custo documentado: ordenação completa onde bastaria seleção parcial.

3.4 Assistentes de prova e orquestração multiagente

Lean 4 (De Moura e Ullrich, 2021) e Mathlib (The Mathlib Community, 2020) fundamentam a verificação; o mecanismo de atenção, (Vaswani et al., 2017). Benchmarks de agentes e orquestração multiagente (*AgentBench* (Liu et al., 2023), *PerspectiveGap* (Sun et al., 2026), recompensa de orquestração (Tsang et al., 2026), arcabouços atencionais (Beckenbauer et al., 2025)) delimitam o contexto externo: nenhum foi executado aqui, e a comparação permanece como trabalho futuro (protocolo proposto na Seção 8).

Situada a matemática local, olhamos a fonte da inspiração para separar o que foi herdado do que foi apenas sugerido.

3.5 O mecanismo de atenção original

No Transformer (Vaswani et al., 2017), cada posição de consulta Q compara-se a chaves K por produto escalado, $A = \text{softmax}(QK^\top/\sqrt{d})$, e agrega valores V por AV . Três ideias importam aqui: (i) escores viram distribuição de probabilidade; (ii) a divisão por $\sqrt{d}$ e a temperatura controlam a nitidez; (iii) múltiplas cabeças capturam aspectos distintos, combinados depois. Exemplo numérico real (implementação deste trabalho): $\text{softmax}([1, 2, 3]) = [0,0900; 0,2447; 0,6652]$, soma 1,0 — o maior escore leva dois terços da massa. A linhagem da atenção inclui abordagens efetivas para tradução neural (Luong, Pham e Manning, 2015), pré-treino bidirecional (Devlin et al., 2019), dependências longas (Dai et al., 2019) e ferramental consolidado (Wolf et al., 2020); benchmarks como GLUE, SQuAD e ImageNet (Wang et al., 2018; Rajpurkar et al., 2016; Deng et al., 2009) fixaram o padrão de avaliação por tarefas que este trabalho adapta em escala sintética. Nosso roteador herda a forma — ponderação normalizada de múltiplas cabeças com decisão pelo máximo — sem herdar pesos treinados, representações ou desempenho: cada cabeça é uma heurística explícita (similaridade, cobertura, confiança, carga), e cada peso da combinação é fixo e auditável.

3.6 Temperatura e calibração

Dividir escores por $\tau > 0$ antes do softmax regula a nitidez: $\tau \rightarrow 0$ concentra no máximo, $\tau \rightarrow \infty$ equaliza. A literatura usa a temperatura para calibrar classificadores (Komisarenko e Kull,

2024; Xuan, Yang e Li, 2025; Xie et al., 2024). Nossa contribuição elementar e verificada (T10): com maximizador único, esfriar aumenta estritamente seu peso — sem limites, sem assintótica, só álgebra de exponenciais.

3.7 Ponto flutuante e certificação

Programas com IEEE-754 exigem análise própria de arredondamento (Bagnara et al., 2019; Murray, 2026). A evidência contempla dois níveis: (i) lema de limite superior da exponencial após deslocamento por majorante (T11), em reais exatos; (ii) teste empírico de propriedades de 20 000 vetores contra a implementação real, com pior desvio de soma $2,2 \times 10^{-16}$ e sem violações da invariância a deslocamento acima da tolerância examinada. Certificação total de floats segue futura.

Da matemática à máquina: como essas ideias viram provas conferidas por computador.

3.8 Verificação formal para leigos

Um assistente de provas é um corretor implacável: cada passo lógico é conferido pelo núcleo até axiomas explícitos. Em Lean 4 (De Moura e Ullrich, 2021), teoremas são programas cujo tipo é a proposição; *táticas* (`simp`, `ring`, `omega`, `linarith`, `exact`) são comandos que transformam objetivos. A Mathlib (The Mathlib Community, 2020) é a biblioteca comunitária com milhares de lemas reusados aqui (somas finitas, exponencial real, máximos). `Sorry` e `admit` são atalhos que afirmam sem provar — bloqueados neste trabalho em duas camadas. `#print axioms` lista os axiomas de cada teorema: todos os onze usam só *propext*, *Classical.choice* e *Quot.sound*, o padrão da matemática formalizada sem atalhos.

3.9 Ler uma prova deste trabalho

Cada prova combina: fatos de positividade (exponencial sempre positiva), reescrita de somas (tirar denominador comum de somatórios), monotonicidade (multiplicar desigualdade por número não negativo preserva o sentido) e linearidade (combinar desigualdades). Exemplo guiado (T2): para mostrar que a média ponderada não passa do máximo M , majoramos cada termo $w_i x_i$ por $w_i M$ (válido pois $w_i \geq 0$), somamos as n desigualdades e fatoramos M para fora da soma, que vale 1 por hipótese. A Tabela 3 resume os recursos utilizados nas provas.

3.10 Táticas de uso frequente

Introduzir e instanciar: `intro` traz hipóteses; `exact` entrega a prova pronta; `apply` ajusta conclusões; `refine` deixa buracos honestos. *Reescrever*: `rw` substitui iguais; `simp` simplifica com a base; `ring` e `omega` decidem anéis e inteiros; `decide` computa verdades decidíveis. *Estruturar*: `cases` e `induction` quebram formas; `constructor` monta pares e existências; `obtain` extrai

Tabela 3 – Táticas usadas nas onze provas (guia de leitura).

Tática/lema	Papel
Finset.sum_pos	positividade de somatórios
Finset.sum_div / sum_mul	reescrita de somas com divisão/multiplicação
div_self / div_lt_div_iff	quocientes: igualdade e desigualdade
mul_le_mul_of_nonneg_left	monotonicidade por não negativos
linarith	combinação linear de desigualdades
field_simp	limpeza de denominadores
Real.exp_*	álgebra da exponencial
Finset.exists_max_image	maximizador finito
take_append_drop / pairwise_append	anatomia do top- k

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

testemunhas. *Fechar*: linarith combina lineares; field_simp limpa denominadores; trivial e assumption encerram o óbvio; contradiction e by_contra exploram o absurdo.

Com teoria e ferramenta postas, situamos o trabalho na literatura — declarando o que foi apenas lido.

3.11 Trabalhos relacionados

Calibração por temperatura. A literatura registra a temperatura como parâmetro de nitidez e calibração: relações entre focal loss, temperature scaling e Scorings próprios, com o hiato de generalização gerando sobreconfiança no teste (Komisarenko e Kull, 2024); aspectos práticos e teóricos em classificação e robustez adversarial (Xuan, Yang e Li, 2025); e escalonamento adaptativo para modelos de linguagem (Xie et al., 2024). Nossa T10 contribui a forma elementar verificada por máquina, sem limites assintóticos. *Ponto flutuante*. Aproximação correta de IEEE-754 para verificação de programas, com arredondamentos, infinitos, NaNs e zeros sinalizados (Bagnara et al., 2019); certificação de robustez via Lipschitz sob execução float, com lacunas float-relevantes para garantias (Murray, 2026). Contribuímos T11 e teste empírico de propriedades, sem reivindicar certificação total. *Benchmarks de agentes*. Avaliação multidimensional de LLMs como agentes em ambientes interativos (Liu et al., 2023); lacuna de perspectiva em prompting de orquestração multiagente (Sun et al., 2026); modelagem de recompensa para orquestradores (Tsang et al., 2026); arcabouço com coordenação auto-emergente inspirada em atenção (Beckenbauer et al., 2025) — convergência independente digna de nota, não validação. Nenhum foi executado aqui. (Síntese limitada aos resumos; leitura integral futura.)

3.12 Fichamento por obra (o que cada uma informa)

Calibração: relações entre perda focal, scaling e propriedade (Komisarenko e Kull, 2024) informam T10; prática e teoria em classificação e robustez (Xuan, Yang e Li, 2025) informam a leitura da Tabela 1; escalonamento adaptativo para linguagem (Xie et al., 2024) informa limites do que não fizemos. *Floats*: aproximação correta de IEEE-754 (Bagnara et al., 2019) informa o desenho do fuzz; certificação via Lipschitz sob floats (Murray, 2026) informa o que faltaria para certificar. *Benchmarks*: avaliação multidimensional (Liu et al., 2023), lacuna de perspectiva (Sun et al., 2026), recompensa de orquestração (Tsang et al., 2026) e coordenação atencional (Beckenbauer et al., 2025) informam o protocolo externo pendente. *Verificação*: base axiomática (Hoare, 1969), Lean (De Moura e Ullrich, 2021), Mathlib (The Mathlib Community, 2020), Z3 (De Moura e Bjørner, 2008) e SymPy (Meurer et al., 2017) informam o núcleo. *Método*: experimentação (Basili, Selby e Hutchens, 1986; Wohlin et al., 2012), casos (Runeson e Höst, 2009), relato (Page et al., 2021), valores-*p* (Wasserstein e Lazar, 2016), estimação (Cumming, 2014), bootstrap (Efron, 1979), reprodutibilidade (Open Science Collaboration, 2015; Collberg e Proebsting, 2016; Ioannidis, 2005), dados e modelos (Geburu et al., 2021; Mitchell et al., 2019), FAIR (Wilkinson et al., 2016) informam desenho, relato e trilha.

Leitura por obra: calibração informa T10; ponto flutuante informa T11 e o fuzz; benchmarks informam o protocolo externo pendente; verificação informa o núcleo Lean; método informa desenho, relato e estatística; reprodutibilidade informa a trilha; dados e modelos documentados informam manifestos. Nenhuma obra foi executada: são referências de contexto, não resultados reivindicados.

Comentário teórico: cada família acima é relacionada na Tabela 2 e no texto a uma lacuna do trabalho (temperatura a T10, ponto flutuante a T11 e ao fuzz, benchmarks ao protocolo externo pendente); registra-se que esses trabalhos foram apenas lidos em resumo, sem execução, e não integram os resultados reivindicados.

4 MÉTODO

A pesquisa combina formalização matemática, avaliação em simulador e inspeção documental dos artefatos locais. As etapas e seus limites são descritos a seguir para permitir a reprodução e distinguir os registros históricos da revisão editorial.

4.1 Provas formais

O arquivo `roteamento_atencao.lean` (Apêndice A) foi processado por `lake env lean` no ambiente registrado, com Lean v4.34.1 e Mathlib rev. v4.34.1. A auditoria `#print axioms` registra as dependências de cada enunciado. O verificador e a interface MCP incluem controles para `sorry`, `admit` e axiomas não declarados. As propriedades formalizadas referem-se a funções sobre os reais; não foi demonstrado um refinamento completo entre o modelo abstrato e todos os componentes da implementação Python.

4.2 Bancada sintética e conjuntos de instâncias

A bancada principal utiliza cinco sementes (11, 22, 33, 44 e 55), cada uma gerando uma piscina de 40 agentes sintéticos e 40 tarefas, totalizando 200 decisões. A qualidade latente q não é fornecida diretamente ao roteador, mas a confiança é construída como uma proxy informativa dessa qualidade. O oráculo atribui utilidade cobertura $\times ((1 - q_w) + q_w q)$. Comparam-se o roteador, a seleção aleatória, a escolha de menor carga e a escolha de maior cobertura. Registram-se arrependimento, acerto, latência e desvio da soma dos pesos. As ablações utilizam as sementes efetivas 10011, 10022, 10033, 10044 e 10055, produzindo outro conjunto de 200 decisões. Incluem média igualitária das cabeças, remoção da confiança e confiança embaralhada, anticorrelacionada ou ruidosa. O pareamento compara estratégias dentro de cada execução das ablações; não vincula essas decisões às instâncias da bancada principal. A coleta de correlações utiliza sua própria sequência de sorteios e também não corresponde integralmente às tarefas das ablações. Na condição de confiança ruidosa, adiciona-se ruído gaussiano com desvio 0,4, limita-se o resultado a $[0, 1]$ e aplica-se arredondamento inteiro, gerando confiança 0 ou 1. O tratamento combina perturbação e discretização binária.

4.3 Parâmetro do oráculo

Nas ablações, avaliam-se $q_w \in \{0,3; 0,6; 0,9\}$ sobre as mesmas instâncias. O filtro de elegibilidade exige cobertura completa; assim, o oráculo entre candidatos elegíveis reduz-se a $1 - q_w + q_w q$. Para $q_w > 0$, seu ranking permanece inalterado e o arrependimento é $q_w(q_{\max} - q_{\text{escolhido}})$. A variação descreve a escala da perda, sem testar critérios ou ordenações diferentes do oráculo.

4.4 Testes numéricos e estatística exploratória

O relatório interno registra testes aleatórios de 20 000 vetores, de tamanho 1–50 e escores de magnitude até ± 1000 , examinando propriedades da implementação em ponto flutuante. A ausência de violações na amostra não certifica todas as entradas nem substitui a prova de refinamento numérico. A documentação da segunda execução mencionada na versão anterior não foi identificada de forma individualizável nesta revisão; o total aqui relatado restringe-se ao registro local identificado. Os intervalos foram calculados por aproximação normal com desvio de divisor n . As 40 decisões de cada semente compartilham uma piscina de agentes. Como os intervalos não consideram esse agrupamento em cinco piscinas, são resumos exploratórios que podem subestimar a incerteza entre piscinas. Uma análise com agrupamento permanece pendente. Correlações de Pearson descrevem associações lineares; não estabelecem causalidade.

4.5 Ameaças à validade e considerações éticas

Construção: o oráculo é arbitrário e a confiança fornece informação sobre a qualidade latente por desenho. A correlação registrada de aproximadamente $+0,85$ explicita essa proxy privilegiada. *Validade interna:* o pareamento controla diferenças de dificuldade entre estratégias de uma mesma execução, mas não elimina todos os fatores de confusão. *Validade externa:* os resultados são restritos ao simulador; não há avaliação executada em benchmarks externos nem mensuração de tarefas realizadas por agentes. *Validade de conclusão:* cinco piscinas, comparadores simples e intervalos sem agrupamento limitam a interpretação inferencial. Não foram utilizados dados de participantes humanos nos experimentos descritos. A declaração ética exigida pelo programa e eventuais dispensas institucionais permanecem sob responsabilidade do autor. O estudo não fundamenta alegações de superioridade externa ou de originalidade validada por avaliadores externos.

4.6 Protocolo de desenvolvimento e reprodução

Os registros de desenvolvimento vinculam as funcionalidades às especificações SPEC-935 e às transcrições de testes preservadas nos apêndices. O protocolo documenta etapas RED–GREEN, auditorias de axiomas e correções de falhas. Esses registros históricos não equivalem à reexecução integral na data desta revisão. As versões, os hashes e as saídas devem acompanhar o depósito final dos artefatos, ainda pendente.

4.7 Interpretação das métricas

O arrependimento é a diferença entre a maior utilidade do oráculo e a utilidade do candidato escolhido; zero indica uma escolha ótima segundo esse oráculo. A taxa de acerto é a fração de decisões que atingem esse máximo. A taxa de vitória ou empate nas ablações é a fração de

decisões em que a atenção apresenta arrependimento menor ou igual ao comparador. Pearson r mede associação linear, sem demonstrar causalidade ou independência quando próximo de zero. A semente fixa a sequência pseudoaleatória utilizada em cada coleta.

4.8 Intervalos registrados e seus limites

O relato por estimação é apoiado por Cumming (2014); Wasserstein e Lazar (2016). Nos scripts deste trabalho, a margem registrada é $1,96s_n/\sqrt{n}$, com s_n calculado com divisor n . Com $n = 200$ e margem 0,0105, obtém-se $s_n \approx 0,0758$. Essa identidade explica o cálculo preservado, sem validar a independência das decisões. A interpretação comparativa requer considerar o desenho, a dependência entre observações e o intervalo da diferença pareada. A não sobreposição de intervalos isolados não substitui essa análise. Como cada piscina gera 40 decisões, a análise futura deve considerar o agrupamento por semente. Reamostragem (Efron, 1979) é uma possibilidade, desde que respeite a unidade de agrupamento; cinco piscinas ainda limitam a precisão. Não foi efetuado recálculo inferencial nesta revisão.

4.9 Pareamento e calibração probabilística

Na ablação com $q_w = 0,6$, a diferença de arrependimento entre média igualitária e atenção é aproximadamente $+0,035$, com margem exploratória 0,011. O sinal positivo descreve vantagem na amostra pareada registrada; não demonstra robustez fora desse desenho. A normalização de pesos não é calibração probabilística. Regras de escore próprias (Gneiting e Raftery, 2007) oferecem fundamentos para outra análise, mas não foram calculadas curvas de calibração, erros de calibração ou escores probabilísticos nesta avaliação.

5 PROVAS FORMAIS

A formalização abrange o núcleo matemático abstrato. Não foi demonstrado um refinamento completo entre esses objetos e a implementação Python, incluindo elegibilidade, representações semânticas, desempate, arredondamento e execução dos agentes. Os registros de auditoria dos onze enunciados são apresentados na Tabela 4. Os enunciados e suas hipóteses são resumidos a seguir: normalização e limitação convexa são o conteúdo formal da exigência de soma unitária dos pesos, prática padrão em ponderação exponencial desde (Vaswani et al., 2017) e objeto de calibração na literatura (Komisarenko e Kull, 2024; Xuan, Yang e Li, 2025); maximizador finito e seleção gulosa formalizam a regra de decisão; invariância, pertinência, singleton, monotonicidade, top- k , nitidez e overflow cobrem estabilidade, casos-limite e decisão, cada qual demonstrado em Lean sobre Mathlib (De Moura e Ullrich, 2021; The Mathlib Community, 2020):

$$\sum_i \left(e^{f_i} / \sum_j e^{f_j} \right) = 1 \quad (\text{T1})$$

$$m \leq \sum_{i \in s} w_i x_i \leq M, \quad \sum_{i \in s} w_i = 1, \quad w_i \geq 0, \quad m \leq x_i \leq M \quad (\text{T2/T6})$$

$$\exists \text{melhor} \in C, \forall a \in C, u(a) \leq u(\text{melhor}) \quad (\text{T3})$$

$$e^{f_i - c} / \sum_j e^{f_j - c} = e^{f_i} / \sum_j e^{f_j} \quad (\text{T4})$$

$$0 < e^{f_i} / \sum_j e^{f_j} < 1 \text{ (2+ candidatos); } = 1 \text{ (singleton)} \quad (\text{T5/T7})$$

$$x \mapsto e^x / (e^x + C) \text{ estritamente crescente para } C > 0 \quad (\text{T8})$$

$$l \text{ pairwise-decrescente} \Rightarrow \forall x \in \text{drop}_k, \forall y \in \text{take}_k, y \geq x \quad (\text{T9})$$

$$0 < \tau_1 < \tau_2 \Rightarrow w_{i^*}(\tau_1) > w_{i^*}(\tau_2) \text{ (maximizador único; ao menos dois candidatos)} \quad (\text{T10})$$

$$f_i \leq M \Rightarrow e^{f_i - M} \leq 1 \quad (\text{T11})$$

Ingredientes de prova, por teorema: (T1) $\sum_i (e^{f_i} / \sum_j e^{f_j}) = 1$ em tipo finito não vazio (positividade da exponencial, `Finset.sum_pos` sobre o universo, reescrita `Finset.sum_div`, `div_self`). (T2/T6) $m \leq \sum w_i x_i \leq M$ sob pesos convexos (monotonicidade da multiplicação por não negativos, reescrita `Finset.sum_mul`, hipótese de soma 1). (T3) $\exists$ maximizador finito (máximo de imagem, `Finset.exists_max_image`). (T4) invariância a $f_i - c$ (subtração da exponencial, soma de quocientes, simplificação de corpos que fecha sozinha). (T5) pesos em $(0, 1)$ sob não-trivialidade (divisão, soma do resto positivo via `add_sum_erase`, `linear`). (T7) peso 1 no singleton (soma de um termo, `sum_eq_single_of_mem` com elemento explícito — cuja ausência gerou `sorryAx` na primeira versão, capturado pela auditoria). (T8) monotonicidade estrita no próprio escore (divisão de desigualdades na forma atual `div_lt_div_iff`, `linear`; nome consultado no Mathlib pinado, não na memória). (T9) take- k domina o resto sob pairwise (concatenação

take/drop, pairwise_append). (T10) nitidez por temperatura: com maximizador único, esfriar aumenta seu peso (fatoração de somas, divisão de desigualdades, soma estrita com testemunha, exigindo não-trivialidade — sem a qual a tese é falsa). (T11) limite superior da exponencial após deslocamento: expoentes deslocados ≤ 0 (caracterização da exponencial real). Auditoria: todos com axiomas restritos a *propext*, *Classical.choice*, *Quot.sound* (Tabela 4), sem *sorryAx*.

Tabela 4 – Auditoria de axiomas (#print axioms).

Teorema	Axiomas
T1 softmax_soma_um	propext, Classical.choice, Quot.sound
T2 combinacao_convexa_limitada	propext, Classical.choice, Quot.sound
T3 roteamento_guloso_otimo	propext, Classical.choice, Quot.sound
T4 softmax_invariante_max	propext, Classical.choice, Quot.sound
T5 softmax_mem_Ioo	propext, Classical.choice, Quot.sound
T6 combinacao_convexa_limitada_baixo	propext, Classical.choice, Quot.sound
T7 softmax_peso_total_singleton	propext, Classical.choice, Quot.sound
T8 peso_cresce_com_escore	propext, Classical.choice, Quot.sound
T9 topk_dominam_resto	propext, Classical.choice, Quot.sound
T10 temperatura_aguca_maximo	propext, Classical.choice, Quot.sound
T11 sem_overflow_apos_max	propext, Classical.choice, Quot.sound

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

5.1 Demonstrações integrais, teorema por teorema

Cada subseção traz enunciado, derivação completa, instância numérica real e necessidade das hipóteses; os nomes de lemas entre parênteses são os usados no código Lean do Apêndice A.

5.1.1 T1 — soma unitária

Enunciado: $\sum_i (e^{f_i} / \sum_j e^{f_j}) = 1$ em tipo finito não vazio. Derivação: cada termo exponencial é positivo (*exp_pos*), logo a soma S sobre o universo não vazio é positiva (*sum_pos*) e não nula; reescreva a soma de quocientes como quociente das somas (*sum_div*); obtém-se $S/S = 1$ (*div_self*). Instância: $f = [1, 2, 3]$ dá pesos $[0,0900; 0,2447; 0,6652]$ somando 1,0. Sem o não-vazio, o denominador seria soma vazia ($= 0$) e a fração indefinida.

5.1.2 T2/T6 — limitação bilateral convexa

Enunciados: $m \leq \sum w_i x_i \leq M$ sob $w_i \geq 0$, $\sum w_i = 1$, $m \leq x_i \leq M$. Derivação: termo a termo, $w_i m \leq w_i x_i \leq w_i M$ (*mul_le_mul_of_nonneg_left*); somar preserva cada lado (*sum_le_sum*); fatorar a constante (*sum_mul*) e usar soma = 1. Instância: $w = [0,3; 0,7]$, $x = [2, 8]$: $6,2 \in [2, 8]$. Contraexemplo sem a hipótese: pesos $[2, -1]$ somando 1 com valores $[0, 10]$ devolvem $-10 < \min$ — por isso o validador rejeita pesos fora de $[0, 1]$.

5.1.3 T3 — maximizador finito

Enunciado: existe maximizador em conjunto finito não vazio. Derivação: invocação direta do máximo de imagem (`exists_max_image`); finitude impede fuga ao infinito, não-vazio garante testemunha. Sem finitude ($\mathbb{N}$ com identidade), não há máximo.

5.1.4 T4 — invariância a deslocamento

Enunciado: subtrair c nada altera. Derivação: $e^{f_j - c} = e^{f_j} / e^c$ (`exp_sub`); soma de quocientes vira quociente das somas; simplificação de corpos fecha sozinha (`field_simp`), pois $e^c \neq 0$. Instância: $[1, 2, 3]$ e $[-4, -3, -2]$ produzem os mesmos pesos em aritmética real; os valores arredondados do exemplo coincidem.

5.1.5 T5 — pertinência a $(0, 1)$

Enunciado: cada peso em $(0, 1)$ com ao menos dois candidatos. Derivação: positividade por divisão de positivos; escreva $S = e^{f_i} + R$ com $R > 0$ (resto via `add_sum_erase`, não vazio por testemunha da não-trivialidade); então $e^{f_i} / S < 1$ por divisão e linearidade. Instância: os três pesos de $[1, 2, 3]$ estão em $(0, 1)$. Sem dois candidatos, o singleton dá peso 1 — caso de T7.

5.1.6 T7 — peso unitário no singleton

Enunciado: candidato único recebe peso 1. Derivação: a soma colapsa no único termo (`sum_eq_single_of_mem` com elemento explícito — cuja ausência gerou `sorryAx`, capturado pela auditoria) e $e/e = 1$. Instância: $[5] \rightarrow [1, 0]$, também observada no roteador real.

5.1.7 T8 — monotonicidade estrita

Enunciado: $x \mapsto e^x / (e^x + C)$ estritamente crescente para $C > 0$. Derivação: reduza a produtos cruzados (divisão de desigualdades na forma vigente, consultada no fonte pinado) e conclua por linearidade com $e^a C < e^b C$. Instância: $C = 1$ dá $0,7311 < 0,8808$. Sem $C > 0$, a fração degenera.

5.1.8 T9 — dominância top- k

Enunciado: numa lista pairwise-decrescente, o prefixo domina o resto. Derivação: reescreva como `take ++ drop`, decomponha a ordem da concatenação e aplique a componente cruzada. É a anatomia do padrão ordenar-e-fatiar, independente do algoritmo de ordenação.

5.1.9 T10 — nitidez por temperatura

Enunciado: com maximizador único, esfriar aumenta seu peso. Derivação: fator $A(\tau_1)S(\tau_2)$ contra $A(\tau_2)S(\tau_1)$; cada termo com $j \neq i^*$ é estrito pela identidade das diferenças positivas;

some com testemunha estrita e conclua por divisão de desigualdades. Instância: $0,6652 \rightarrow 0,8668$. O enunciado formal adota maximizador único e pelo menos dois candidatos como hipóteses suficientes. Máximos empatados podem ganhar massa quando existem escores inferiores; se todos os escores forem iguais, a temperatura não altera os pesos.

5.1.10 T11 — limite superior da exponencial após deslocamento

Enunciado: se $f_i \leq M$, então $e^{f_i - M} \leq 1$ em reais exatos. Derivação: monotonicidade da exponencial e $e^0 = 1$. A prova não certifica subtração, exponenciação, soma ou divisão IEEE-754. Instâncias: $e^{-2} \approx 0,1353$ e $e^0 = 1,0$. Sem majorante, nada limita os expoentes.

Mapa de dependências: T1 sustenta T5 e T7; T2/T6 sustentam a combinação das cabeças; T4 sustenta a implementação estável usada em T10; T3 sustenta a decisão por máximo ao lado de T8; T9 descreve a ordenação do top- k , sem constituir sozinho prova do algoritmo concreto.

5.2 Necessidade das hipóteses (contraexemplos)

Reunidas: T1 exige não-vazio; T2 exige não-negatividade e soma 1; T3 exige finitude e não-vazio; T5 exige dois candidatos; T8 exige $C > 0$; T10 exige maximizador único e não-trivialidade; T11 exige majorante. As hipóteses delimitam os enunciados formalizados. Os contraexemplos apresentados mostram falhas específicas quando algumas delas são removidas; não demonstram a necessidade individual de todas as hipóteses.

6 AVALIAÇÃO EMPÍRICA E TESTES NUMÉRICOS

Esta seção apresenta os resultados históricos da bancada e das ablações, distinguidos pelos conjuntos de instâncias descritos na Seção 4.

6.1 Fuzz em ponto flutuante

O relatório interno registra testes aleatórios de 20 000 vetores, sem violações acima das tolerâncias examinadas: 10^{-9} para a soma e 10^{-12} para o deslocamento. O maior desvio registrado da soma foi aproximadamente $2,2 \times 10^{-16}$. A preservação de argmax, os limites, o caso de candidato único e o tratamento do conjunto vazio também foram examinados. A ausência de violações nessa amostra não demonstra correção para todas as entradas. Testes por entradas aleatórias têm antecedentes na avaliação de utilitários Unix (Miller, Fredriksen e So, 1990).

6.2 Bancada principal e variação do parâmetro do oráculo

Bancada principal (200 decisões): atenção $0,042 \pm 0,011$ (acerto $0,625 \pm 0,067$) contra 0,19–0,22; latência 2,17 ms; soma softmax 1 até $1,1 \times 10^{-16}$. Os valores da bancada principal constam de `resultado_bancada.json`, reproduzido no Apêndice D. A Tabela 5 apresenta as médias das ablações para cada valor do parâmetro. Outras condições de ablação ($q_w = 0,6$): confiança

Tabela 5 – Ablações: arrependimento por parâmetro do oráculo e diferença em relação à atenção (200 decisões por configuração).

q_w	Atenção	Pesos iguais (dif.)	Sem confiança (dif.)
0,3	0,0146	0,0319 (+0,017)	0,1072 (+0,093)
0,6	0,0292	0,0638 (+0,035)	0,2144 (+0,185)
0,9	0,0438	0,0957 (+0,052)	0,3216 (+0,278)

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

embaralhada 0,2243 (+0,195), anticorrelacionada 0,3604 (+0,331), ruidosa 0,1515 (+0,122); vitória/empate da atenção $\geq 0,92$ em todos os pareamentos. Os registros internos relatam reexecução determinística; uma reprodução independente permanece pendente. A Tabela 6 reúne as diferenças pareadas e as taxas de vitória ou empate. O ranking das estratégias permanece estável na Tabela 7 porque, entre elegíveis com cobertura unitária, o arrependimento é proporcional a q_w . As configurações não representam avaliações independentes de oráculos distintos. As diferenças pareadas e suas margens descrevem as instâncias das ablações, sem correção para agrupamento. A atenção obteve o menor arrependimento nas 5 sementes avaliadas (Tabela 8).

Tabela 6 – Diferenças pareadas por decisão vs. atenção ($q_w = 0,6$) e taxa de vitória/empate.

Estratégia	Diferença $\pm$ margem exploratória	Atenção vence/empata
Média igualitária	$+0,035 \pm 0,011$	0,99
Sem confiança	$+0,185 \pm 0,024$	0,98
Confiança ruidosa	$+0,122 \pm 0,022$	0,93
Confiança embaralhada	$+0,195 \pm 0,025$	0,92
Confiança anticorrelacionada	$+0,331 \pm 0,019$	0,97

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

Tabela 7 – Arrependimento por estratégia nos três valores do parâmetro do oráculo (200 decisões cada).

Estratégia	$q_w = 0,3$	$q_w = 0,6$	$q_w = 0,9$
Atenção	0,0146	0,0292	0,0438
Média igualitária	0,0319	0,0638	0,0957
Sem confiança	0,1072	0,2144	0,3216
Confiança ruidosa	0,0757	0,1515	0,2272
Confiança embaralhada	0,1121	0,2243	0,3364
Confiança anticorrelacionada	0,1802	0,3604	0,5406

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

Tabela 8 – Arrependimento por semente e estratégia (40 decisões cada; acerto da atenção entre parênteses).

Semente	Atenção	Aleatório	Menor carga	Maior cobertura
11	0,0752 (0,45)	0,1889	0,3157	0,2319
22	0,0172 (0,78)	0,2463	0,2276	0,2310
33	0,0337 (0,55)	0,1719	0,2447	0,2033
44	0,0716 (0,60)	0,1992	0,2038	0,2309
55	0,0129 (0,75)	0,1350	0,0918	0,1618

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

6.3 Estudo de caso completo: Ana contra Cid

Na tarefa-guia, as cabeças mediram: semântica Ana 0,8417 contra Cid 0,7703; capacidade 1,0 para ambas (o gate já havia decidido); confiança 0,9 contra 0,7; carga livre 0,9 contra 0,6. Utilidades: Ana $0,30 \cdot 0,8417 + 0,35 \cdot 1,0 + 0,25 \cdot 0,9 + 0,10 \cdot 0,9 = 0,9175$; Cid $0,30 \cdot 0,7703 + 0,35 \cdot 1,0 + 0,25 \cdot 0,7 + 0,10 \cdot 0,6 = 0,8161$. Pesos finais 0,5253 contra 0,4747. Note-se a capacidade constante: com cobertura total dos elegíveis, a decisão correu nas outras três cabeças — exatamente o fenômeno quantificado nas correlações.

6.4 Correlações de Pearson (200 decisões)

Arrependimento $\times$ elegíveis +0,08; arrependimento $\times$ dispersão -0,16; acerto $\times$ elegíveis -0,19; acerto $\times$ dispersão +0,13; confiança $\times q$ +0,85 (checagem de desenho: proxy forte, fonte da circularidade parcial); cabeças quase descorrelacionadas (semântica $\times$ confiança +0,05); capacidade constante após o filtro de elegibilidade (o gate decide cobertura; a escolha corre nas outras cabeças). A transcrição histórica preservada registra 63 testes aprovados em nove suítes. A revisão editorial não reexecutou essa bateria. O diagnóstico preliminar da revisão apresentou 18 verificações aprovadas e duas advertências, sem falhas.

6.5 Demonstração de roteamento sobre cartões do catálogo

Foram utilizados onze cartões extraídos do catálogo local para cinco descrições de tarefas construídas com seu vocabulário de tags. A demonstração avalia elegibilidade e ranking; não executa os agentes nem mede o sucesso na realização das tarefas. Quatro decisões foram singleton (peso 1,0 — T7 em operação); a quinta opôs reviewer a code-reviewer em tarefa de revisão: ambas com cobertura, confiança e carga idênticas (1,0; 0,5; 1,0), a semântica decidiu 0,5589 e o placar fechou 0,505 contra 0,495 — decisão puramente semântica, à margem mínima. A primeira decisão custou 27,8 ms (carga fria do casamento semântico) e as seguintes 0,4–0,5 ms; soma dos pesos 1,0 em todas. Tarefas fora do vocabulário retornam vazio por gate — resultado compatível com o contrato do filtro; a cobertura do vocabulário do catálogo ainda requer avaliação.

6.6 Interpretação das estratégias

A estratégia de atenção utiliza quatro componentes com pesos fixos declarados, sem aprendizagem desses pesos. No simulador, a confiança é construída a partir da qualidade latente com ruído, constituindo uma proxy informativa por desenho. A média igualitária compara outra ponderação dos mesmos componentes; a remoção da confiança avalia a decisão sem essa informação. A seleção aleatória sorteia entre elegíveis. A escolha de menor carga utiliza a disponibilidade, que não compõe diretamente a qualidade latente do oráculo. A escolha de maior cobertura enfrenta empate entre candidatos plenamente elegíveis, pois o filtro já exigiu cobertura completa.

Seu resultado não mede discriminação adicional por cobertura. A remoção da confiança e suas perturbações produziram maior arrependimento nas amostras das ablações. Esse resultado é compatível com a dependência da decisão em relação à proxy. Não demonstra equivalência estatística com o acaso, relação dose–resposta ou ausência de influência das sementes. A condição ruidosa inclui discretização binária, conforme descrito na Seção 4.

7 DISCUSSÃO

Os resultados articulam duas formas de evidência com alcances diferentes: as provas documentam propriedades do modelo matemático; a bancada descreve o comportamento de uma implementação em instâncias sintéticas. A composição torna explícitos os pressupostos da decisão, mas não fornece uma certificação integral da execução nem uma comparação de desempenho em tarefas reais.

7.1 Alcance das garantias formais

Os onze lemas são propriedades matemáticas elementares, formalizadas sobre o objeto estudado. Não se reivindica originalidade desses fatos. T3 estabelece existência de maximizador finito, enquanto T9 descreve a dominância de um prefixo de lista já ordenada. A correspondência entre essas propriedades e o código exige demonstrar a preservação de hipóteses pelo filtro, pelos escores, pelo desempate e pela aritmética utilizada. T11 limita a exponencial deslocada em reais exatos; erros de arredondamento, underflow, entradas não finitas e operações intermediárias permanecem fora dessa prova.

7.2 Dependência do desenho sintético

A vantagem observada depende da confiança, construída como proxy da qualidade latente. Sua remoção aumenta o arrependimento no conjunto de ablações, sem estabelecer equivalência com a seleção aleatória. Mesmo confiança igual à qualidade latente não garantiria que a utilidade combinada preservasse o ranking do oráculo, pois semântica e carga também influenciam a escolha. Essa configuração não foi avaliada. Os três valores de q_w reescalam a perda sem mudar o ranking do oráculo entre elegíveis. Sua estabilidade não constitui evidência independente de robustez a outros critérios de utilidade. Os intervalos preservados ignoram a dependência dentro das piscinas; uma análise que considere agrupamento pode alterar as margens e a interpretação comparativa.

7.3 Riscos residuais e avaliação interna

Duas revisões internas automatizadas examinaram a bancada e os teoremas. Não constituem revisão cega ou validação externa. As objeções sobre proxy, comparadores, analogia Q/K/V e lacuna numérica são compatíveis com as limitações descritas na Tabela 9. As ablações acrescentam comparações entre ponderações e perturbações, mas não eliminam essas objeções.

Tabela 9 – Limitações identificadas e requisitos de avaliação futura.

Limitação	Requisito futuro
Oráculo arbitrário e proxy privilegiada	Outros critérios de qualidade e confiança sem acesso ao gerador latente
Cinco piscinas e intervalos sem agrupamento	Mais piscinas independentes e análise na unidade correta
Modelo real e implementação em ponto flutuante	Refinamento formal e tratamento de casos numéricos especiais
Comparadores internos	Benchmark externo executado com orçamento e critérios comuns
Artefatos locais	Depósito versionado e reprodução independente

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

7.4 Relação com a literatura e perspectivas

A literatura de calibração por temperatura (Komisarenko e Kull, 2024; Xuan, Yang e Li, 2025; Xie et al., 2024) aborda objetivos que não foram medidos aqui: a normalização por softmax não demonstra calibração probabilística. Os estudos de aritmética flutuante (Bagnara et al., 2019; Murray, 2026) indicam a necessidade de garantias próprias para a execução numérica. Os benchmarks de agentes (Liu et al., 2023; Sun et al., 2026; Tsang et al., 2026; Beckenbauer et al., 2025) fornecem contextos para comparação futura, sem validar os resultados locais. A interpretação educacional das trilhas auditáveis é uma hipótese de aplicação futura, sem avaliação com estudantes nesta pesquisa.

8 CONCLUSÃO

Retomando a questão da Seção 1, registramos o que foi demonstrado, o que foi medido e o que permanece aberto. A questão da pesquisa foi respondida em dois níveis: os registros documentam onze lemas do modelo matemático formalizados em Lean 4 e uma avaliação preliminar da implementação no simulador, com artefatos locais de reprodução. Reivindica-se otimalidade relativa e evidência preliminar. Contribuições numeradas: (C1) onze lemas com provas verificadas por máquina e axiomas auditados; (C2) pipeline de verificação lake-env com bloqueio a atalhos; (C3) bancada com intervalos exploratórios, pareamento e variação de escala do oráculo; (C4) testes aleatórios de propriedades numéricas e auditoria de alegações e referências; (C5) trilha de reprodução com pinos, hashes e protocolo de sincronia. Trabalhos futuros: Q/K/V formal completo, erro numérico certificado à Flocq, análise com agrupamento por piscina, temperatura com limites, benchmark externo com suíte compartilhada e orçamento próprio. Os objetivos de formalização (OE1), comparação interna (OE2) e auditoria documental (OE3) são sustentados pelos registros locais nos limites explicitados. O objetivo de disponibilização pública (OE4) permanece parcialmente atendido: os artefatos estão organizados localmente, mas o depósito público e sua identificação persistente estão pendentes. A Tabela 10 sintetiza os requisitos de continuidade da pesquisa.

Tabela 10 – Programa de trabalhos futuros (pacotes com aceite).

Pacote	Método	Aceite
Q/K/V formal	Estruturas + refinamento	machine-checked
Erro certificado	Intervalos + Flocq	cotas verificadas
Temperatura limite	Análise assintótica	teoremas de limite
Benchmark externo	Suíte + orçamento	head-to-head executado
Reprodução independente	Terceiros + pins	mesmos números
Seleção parcial	Heap + benchmark	mesmos top- k , menor latência

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

REFERÊNCIAS

- BAGNARA, R. et al. *Correct Approximation of IEEE 754 Floating-Point Arithmetic for Program Verification*. [S. l.]: arXiv, 2019. Preprint arXiv:1903.06119. DOI: 10.48550/arXiv.1903.06119. Disponível em: <https://arxiv.org/abs/1903.06119>. Acesso em: 10 out. 2026.
- BASILI, V. R.; SELBY, R. W.; HUTCHENS, D. H. Experimentation in software engineering. *IEEE Transactions on Software Engineering*, [s. l.], v. SE-12, n. 7, p. 733–743, 1986. DOI: 10.1109/TSE.1986.6312975. Disponível em: <https://doi.org/10.1109/TSE.1986.6312975>. Acesso em: 10 out. 2026.
- BECKENBAUER, L. et al. *Orchestrator: Active Inference for Multi-Agent Systems in Long-Horizon Tasks*. [S. l.]: arXiv, 2025. Preprint arXiv:2509.05651. DOI: 10.48550/arXiv.2509.05651. Disponível em: <https://arxiv.org/abs/2509.05651>. Acesso em: 10 out. 2026.
- BRIDLE, J. S. Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition. In: SOULIÉ, F. F.; HÉRAULT, J. (ed.). *Neurocomputing*. Berlin; Heidelberg: Springer, 1990. p. 227–236. (NATO ASI Series, v. 68). DOI: 10.1007/978-3-642-76153-9_28. Disponível em: https://doi.org/10.1007/978-3-642-76153-9_28. Acesso em: 10 out. 2026.
- CHIDAMBER, S. R.; KEMERER, C. F. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, [s. l.], v. 20, n. 6, p. 476–493, 1994. DOI: 10.1109/32.295895. Disponível em: <https://doi.org/10.1109/32.295895>. Acesso em: 10 out. 2026.
- COLLBERG, C.; PROEBSTING, T. A. Repeatability in computer systems research. *Communications of the ACM*, [s. l.], v. 59, n. 3, p. 62–69, 2016. DOI: 10.1145/2812803. Disponível em: <https://doi.org/10.1145/2812803>. Acesso em: 10 out. 2026.
- CUMMING, G. The New Statistics: Why and How. *Psychological Science*, [s. l.], v. 25, n. 1, p. 7–29, 2014. DOI: 10.1177/0956797613504966. Disponível em: <https://doi.org/10.1177/0956797613504966>. Acesso em: 10 out. 2026.
- DAI, Z. et al. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 57., 2019, Florence. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence: Association for Computational Linguistics, 2019. p. 2978–2988. DOI: 10.18653/v1/P19-1285. Disponível em: <https://doi.org/10.18653/v1/P19-1285>. Acesso em: 10 out. 2026.
- DE MOURA, L.; BJØRNER, N. Z3: An Efficient SMT Solver. In: INTERNATIONAL CONFERENCE ON TOOLS AND ALGORITHMS FOR THE CONSTRUCTION AND

ANALYSIS OF SYSTEMS, 14., 2008, Budapest. *Tools and Algorithms for the Construction and Analysis of Systems*. Berlin; Heidelberg: Springer, 2008. Lecture Notes in Computer Science, v. 4963. p. 337–340. DOI: 10.1007/978-3-540-78800-3_24. Disponível em: https://doi.org/10.1007/978-3-540-78800-3_24. Acesso em: 10 out. 2026.

DE MOURA, L.; ULLRICH, S. The Lean 4 Theorem Prover and Programming Language. In: INTERNATIONAL CONFERENCE ON AUTOMATED DEDUCTION, 28., 2021, evento virtual. *Automated Deduction – CADE 28*. Cham: Springer, 2021. Lecture Notes in Computer Science, v. 12699. p. 625–635. DOI: 10.1007/978-3-030-79876-5_37. Disponível em: https://doi.org/10.1007/978-3-030-79876-5_37. Acesso em: 10 out. 2026.

DENG, J. et al. ImageNet: A large-scale hierarchical image database. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION, 2009, Miami. *2009 IEEE Conference on Computer Vision and Pattern Recognition*. [S. l.]: IEEE, 2009. p. 248–255. DOI: 10.1109/CVPR.2009.5206848. Disponível em: <https://doi.org/10.1109/CVPR.2009.5206848>. Acesso em: 10 out. 2026.

DEVLIN, J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: CONFERENCE OF THE NORTH AMERICAN CHAPTER OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS: HUMAN LANGUAGE TECHNOLOGIES, 2019, Minneapolis. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis: Association for Computational Linguistics, 2019. p. 4171–4186. DOI: 10.18653/v1/N19-1423. Disponível em: <https://doi.org/10.18653/v1/N19-1423>. Acesso em: 10 out. 2026.

EFRON, B. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, [s. l.], v. 7, n. 1, p. 1–26, 1979. DOI: 10.1214/aos/1176344552. Disponível em: <https://doi.org/10.1214/aos/1176344552>. Acesso em: 10 out. 2026.

ERMAN, L. D. et al. The Hearsay-II Speech-Understanding System: Integrating Knowledge to Resolve Uncertainty. *ACM Computing Surveys*, [s. l.], v. 12, n. 2, p. 213–253, 1980. DOI: 10.1145/356810.356816. Disponível em: <https://doi.org/10.1145/356810.356816>. Acesso em: 10 out. 2026.

FAGAN, M. E. Design and code inspections to reduce errors in program development. *IBM Systems Journal*, [s. l.], v. 15, n. 3, p. 182–211, 1976. DOI: 10.1147/sj.153.0182. Disponível em: <https://doi.org/10.1147/sj.153.0182>. Acesso em: 10 out. 2026.

GEBRU, T. et al. Datasheets for datasets. *Communications of the ACM*, [s. l.], v. 64, n. 12, p. 86–92, 2021. DOI: 10.1145/3458723. Disponível em: <https://doi.org/10.1145/3458723>. Acesso em: 10 out. 2026.

GNEITING, T.; RAFTERY, A. E. Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association*, [s. l.], v. 102, n. 477, p. 359–378, 2007. DOI: 10.1198/016214506000001437. Disponível em:

<https://doi.org/10.1198/016214506000001437>. Acesso em: 10 out. 2026.

GOLDBERG, D. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, [s. l.], v. 23, n. 1, p. 5–48, 1991. DOI: 10.1145/103162.103163. Disponível em: <https://doi.org/10.1145/103162.103163>. Acesso em: 10 out. 2026.

HARRIS, C. R. et al. Array programming with NumPy. *Nature*, [s. l.], v. 585, n. 7825, p. 357–362, 2020. DOI: 10.1038/s41586-020-2649-2. Disponível em: <https://doi.org/10.1038/s41586-020-2649-2>. Acesso em: 10 out. 2026.

HOARE, C. A. R. An axiomatic basis for computer programming. *Communications of the ACM*, [s. l.], v. 12, n. 10, p. 576–580, 1969. DOI: 10.1145/363235.363259. Disponível em: <https://doi.org/10.1145/363235.363259>. Acesso em: 10 out. 2026.

IOANNIDIS, J. P. A. Why Most Published Research Findings Are False. *PLoS Medicine*, [s. l.], v. 2, n. 8, e124, 2005. DOI: 10.1371/journal.pmed.0020124. Disponível em: <https://doi.org/10.1371/journal.pmed.0020124>. Acesso em: 10 out. 2026.

JAYNES, E. T. Information Theory and Statistical Mechanics. *Physical Review*, [s. l.], v. 106, n. 4, p. 620–630, 1957. DOI: 10.1103/PhysRev.106.620. Disponível em: <https://doi.org/10.1103/PhysRev.106.620>. Acesso em: 10 out. 2026.

KOMISARENKO, V.; KULL, M. *Improving Calibration by Relating Focal Loss, Temperature Scaling, and Properness*. [S. l.]: arXiv, 2024. Preprint arXiv:2408.11598. DOI: 10.48550/arXiv.2408.11598. Disponível em: <https://arxiv.org/abs/2408.11598>. Acesso em: 10 out. 2026.

KULLBACK, S.; LEIBLER, R. A. On Information and Sufficiency. *The Annals of Mathematical Statistics*, [s. l.], v. 22, n. 1, p. 79–86, 1951. DOI: 10.1214/aoms/1177729694. Disponível em: <https://doi.org/10.1214/aoms/1177729694>. Acesso em: 10 out. 2026.

LIU, X. et al. *AgentBench: Evaluating LLMs as Agents*. [S. l.]: arXiv, 2023. Preprint arXiv:2308.03688. DOI: 10.48550/arXiv.2308.03688. Disponível em: <https://arxiv.org/abs/2308.03688>. Acesso em: 10 out. 2026.

LUONG, T.; PHAM, H.; MANNING, C. D. Effective Approaches to Attention-based Neural Machine Translation. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING, 2015, Lisbon. *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. Lisbon: Association for Computational Linguistics, 2015. p. 1412–1421. DOI: 10.18653/v1/D15-1166. Disponível em: <https://doi.org/10.18653/v1/D15-1166>. Acesso em: 10 out. 2026.

MCCABE, T. J. A Complexity Measure. *IEEE Transactions on Software Engineering*, [s. l.], v. SE-2, n. 4, p. 308–320, 1976. DOI: 10.1109/TSE.1976.233837. Disponível em: <https://doi.org/10.1109/TSE.1976.233837>. Acesso em: 10 out. 2026.

MEURER, A. et al. SymPy: symbolic computing in Python. *PeerJ Computer Science*, [s. l.], v. 3, e103, 2017. DOI: 10.7717/peerj-cs.103. Disponível em: <https://doi.org/10.7717/peerj-cs.103>. Acesso em: 10 out. 2026.

MILLER, B. P.; FREDRIKSEN, L.; SO, B. An empirical study of the reliability of UNIX utilities. *Communications of the ACM*, [s. l.], v. 33, n. 12, p. 32–44, 1990. DOI: 10.1145/96267.96279. Disponível em: <https://doi.org/10.1145/96267.96279>. Acesso em: 10 out. 2026.

MITCHELL, M. et al. Model Cards for Model Reporting. In: CONFERENCE ON FAIRNESS, ACCOUNTABILITY, AND TRANSPARENCY, 2019, Atlanta. *Proceedings of the Conference on Fairness, Accountability, and Transparency*. New York: ACM, 2019. p. 220–229. DOI: 10.1145/3287560.3287596. Disponível em: <https://doi.org/10.1145/3287560.3287596>. Acesso em: 10 out. 2026.

MULLER, J.-M. et al. *Handbook of Floating-Point Arithmetic*. 2. ed. Cham: Birkhäuser, 2018. DOI: 10.1007/978-3-319-76526-6. Disponível em: <https://doi.org/10.1007/978-3-319-76526-6>. Acesso em: 10 out. 2026.

MURRAY, T. *Lipschitz-Based Robustness Certification Under Floating-Point Execution*. [S. l.]: arXiv, 2026. Preprint arXiv:2603.13334. DOI: 10.48550/arXiv.2603.13334. Disponível em: <https://arxiv.org/abs/2603.13334>. Acesso em: 10 out. 2026.

OPEN SCIENCE COLLABORATION. Estimating the reproducibility of psychological science. *Science*, [s. l.], v. 349, n. 6251, aac4716, 2015. DOI: 10.1126/science.aac4716. Disponível em: <https://doi.org/10.1126/science.aac4716>. Acesso em: 10 out. 2026.

PAGE, M. J. et al. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, [s. l.], v. 372, n71, 2021. DOI: 10.1136/bmj.n71. Disponível em: <https://doi.org/10.1136/bmj.n71>. Acesso em: 10 out. 2026.

PARNAS, D. L.; CLEMENTS, P. C. A rational design process: How and why to fake it. *IEEE Transactions on Software Engineering*, [s. l.], v. SE-12, n. 2, p. 251–257, 1986. DOI: 10.1109/TSE.1986.6312940. Disponível em: <https://doi.org/10.1109/TSE.1986.6312940>. Acesso em: 10 out. 2026.

RAJPURKAR, P. et al. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING, 2016, Austin. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Austin: Association for Computational Linguistics, 2016. p. 2383–2392. DOI: 10.18653/v1/D16-1264. Disponível em: <https://doi.org/10.18653/v1/D16-1264>. Acesso em: 10 out. 2026.

RUNESON, P.; HÖST, M. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, [s. l.], v. 14, n. 2, p. 131–164, 2009. DOI: 10.1007/s10664-008-9102-8. Disponível em:

<https://doi.org/10.1007/s10664-008-9102-8>. Acesso em: 10 out. 2026.

SHANNON, C. E. A Mathematical Theory of Communication. *Bell System Technical Journal*, [s. l.], v. 27, n. 3, p. 379–423, 1948. DOI: 10.1002/j.1538-7305.1948.tb01338.x. Disponível em: <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>. Acesso em: 10 out. 2026.

SMITH, R. G. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, [s. l.], v. C-29, n. 12, p. 1104–1113, 1980. DOI: 10.1109/TC.1980.1675516. Disponível em: <https://doi.org/10.1109/TC.1980.1675516>. Acesso em: 10 out. 2026.

SUN, Y. et al. *PerspectiveGap: A Benchmark for Multi-Agent Orchestration Prompting*. [S. l.]: arXiv, 2026. Preprint arXiv:2606.08878. DOI: 10.48550/arXiv.2606.08878. Disponível em: <https://arxiv.org/abs/2606.08878>. Acesso em: 10 out. 2026.

THE MATHLIB COMMUNITY. The lean mathematical library. In: ACM SIGPLAN INTERNATIONAL CONFERENCE ON CERTIFIED PROGRAMS AND PROOFS, 9., 2020, New Orleans. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. New York: ACM, 2020. p. 367–381. DOI: 10.1145/3372885.3373824. Disponível em: <https://doi.org/10.1145/3372885.3373824>. Acesso em: 10 out. 2026.

TSANG, K. Y. et al. *Reward Modeling for Multi-Agent Orchestration*. [S. l.]: arXiv, 2026. Preprint arXiv:2606.13598. DOI: 10.48550/arXiv.2606.13598. Disponível em: <https://arxiv.org/abs/2606.13598>. Acesso em: 10 out. 2026.

VASWANI, A. et al. Attention is all you need. In: ANNUAL CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 31., 2017, Long Beach. *Advances in Neural Information Processing Systems*. [S. l.]: Curran Associates, 2017. v. 30. p. 5998–6008. Disponível em: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html. Acesso em: 10 out. 2026.

VIRTANEN, P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, [s. l.], v. 17, n. 3, p. 261–272, 2020. DOI: 10.1038/s41592-019-0686-2. Disponível em: <https://doi.org/10.1038/s41592-019-0686-2>. Acesso em: 10 out. 2026.

WANG, A. et al. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In: EMNLP WORKSHOP BLACKBOXNLP: ANALYZING AND INTERPRETING NEURAL NETWORKS FOR NLP, 2018, Brussels. *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Brussels: Association for Computational Linguistics, 2018. p. 353–355. DOI: 10.18653/v1/W18-5446. Disponível em: <https://doi.org/10.18653/v1/W18-5446>. Acesso em: 10 out. 2026.

WASSERSTEIN, R. L.; LAZAR, N. A. The ASA Statement on *p*-Values: Context, Process, and Purpose. *The American Statistician*, [s. l.], v. 70, n. 2, p. 129–133, 2016. DOI: 10.1080/00031305.2016.1154108. Disponível em:

<https://doi.org/10.1080/00031305.2016.1154108>. Acesso em: 10 out. 2026.

WILKINSON, M. D. et al. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, [s. l.], v. 3, n. 1, 160018, 2016. DOI: 10.1038/sdata.2016.18. Disponível em: <https://doi.org/10.1038/sdata.2016.18>. Acesso em: 10 out. 2026.

WOHLIN, C. et al. *Experimentation in Software Engineering*. Berlin; Heidelberg: Springer, 2012. DOI: 10.1007/978-3-642-29044-2. Disponível em: <https://doi.org/10.1007/978-3-642-29044-2>. Acesso em: 10 out. 2026.

WOLF, T. et al. Transformers: State-of-the-Art Natural Language Processing. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING: SYSTEM DEMONSTRATIONS, 2020, evento virtual. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. [S. l.]: Association for Computational Linguistics, 2020. p. 38–45. DOI: 10.18653/v1/2020.emnlp-demos.6. Disponível em: <https://doi.org/10.18653/v1/2020.emnlp-demos.6>. Acesso em: 10 out. 2026.

XIE, J. et al. *Calibrating Language Models with Adaptive Temperature Scaling*. [S. l.]: arXiv, 2024. Preprint arXiv:2409.19817. DOI: 10.48550/arXiv.2409.19817. Disponível em: <https://arxiv.org/abs/2409.19817>. Acesso em: 10 out. 2026.

XUAN, H.; YANG, B.; LI, X. *Exploring the Impact of Temperature Scaling in Softmax for Classification and Adversarial Robustness*. [S. l.]: arXiv, 2025. Preprint arXiv:2502.20604. DOI: 10.48550/arXiv.2502.20604. Disponível em: <https://arxiv.org/abs/2502.20604>. Acesso em: 10 out. 2026.

GLOSSÁRIO

Ablação.

Alteração ou remoção de um componente para comparar seu efeito dentro de um desenho definido.

Agente.

Programa ao qual uma tarefa pode ser atribuída; o estudo com cartões calcula rankings e não executa os agentes.

Arrependimento.

Diferença entre a melhor utilidade do oráculo e a utilidade da escolha realizada.

Axioma.

Proposição admitida pelo sistema formal. A auditoria registra `propext`, `Classical.choice` e `Quot.sound`, sem demonstrar minimalidade desse conjunto.

Calibração probabilística.

Correspondência entre probabilidades previstas e frequências observadas; não é garantida apenas pela soma unitária do softmax.

Combinação convexa.

Soma ponderada com pesos não negativos de soma unitária.

Comparador.

Estratégia utilizada como referência para avaliação no mesmo desenho.

Intervalo exploratório.

Faixa calculada pela aproximação normal registrada, sem correção para agrupamento nas piscinas deste estudo.

Oráculo.

Critério sintético usado para avaliar as escolhas; não representa qualidade observada em tarefas reais.

Overflow.

Excesso da faixa representável em ponto flutuante. T11 prova um limite da exponencial em reais exatos e não certifica todas as operações da implementação.

Reprodução.

Execução dos procedimentos por outro executor a partir dos artefatos e do ambiente especificado; reprodução independente permanece pendente.

Roteador.

Mecanismo que filtra candidatos e ordena suas utilidades para selecionar um agente.

Semente.

Valor que inicializa uma sequência pseudoaleatória.

Softmax.

Normalização exponencial de uma família finita não vazia de escores. Em reais exatos, seus pesos somam um e são positivos; em ponto flutuante, underflow pode produzir zeros.

Temperatura.

Parâmetro positivo que controla a concentração dos pesos. T10 utiliza pelo menos dois candidatos e maximizador único como hipóteses suficientes.

Top- k .

Prefixo de tamanho k de um ranking; T9 pressupõe a ordenação decrescente da lista.

Underflow.

Resultado muito pequeno para ser representado com a precisão ou a faixa disponíveis.

Verificação por máquina.

Checagem de um termo de prova pelo núcleo do assistente, relativa ao enunciado e aos axiomas utilizados; não garante que o enunciado modele integralmente o sistema concreto.

APÊNDICE A – CÓDIGO LEAN INTEGRAL

```

-- Teorema de Eficiência do Roteamento por Atenção – SPEC-935-R755
-- Espelha transformer/attention.py: softmax estável, cabeças em combinação
-- convexa (soma 1), ranking top-k. Eficiência = otimalidade relativa à
-- utilidade definida, nunca superioridade universal.

import Mathlib

open BigOperators Finset

-- T1: pesos de atenção (softmax) somam 1 – calibração (attention.py: pesos somando 1).
theorem softmax_soma_um ι { : Type*} [DecidableEq ι] [Fintype ι] (f : ι → Real) [Nonempty ι]
  :
    ∑ i, (Real.exp (f i) / ∑ j, Real.exp (f j)) = 1 := by
  have hpos : 0 < ∑ j, Real.exp (f j) :=
    Finset.sum_pos (fun i _ => Real.exp_pos (f i)) Finset.univ_nonempty
  rw <[ Finset.sum_div ]
  exact div_self (ne_of_gt hpos)

-- T2: combinação convexa das cabeças fica limitada pelo melhor componente
-- (attention.py: _validated_weights exige soma 1, cada peso em [0,1]).
theorem combinacao_convexa_limitada ι { : Type*} (s : Finset ι) (w x : ι → Real) (M : Real)
  (hsum : Finset.sum s (fun i => w i) = 1) (hnn : ∀ i ∈ s, 0 ≤ w i) (hle : ∀ i ∈ s, x i ≤
  M) :
    Finset.sum s (fun i => w i * x i) ≤ M := by
  calc Finset.sum s (fun i => w i * x i) ≤ Finset.sum s (fun i => w i * M) :=
    Finset.sum_le_sum (fun i hi => mul_le_mul_of_nonneg_left (hle i hi) (hnn i hi))
  _ = M := by rw <[ Finset.sum_mul, hsum, one_mul ]

-- T3: existe candidato de utilidade máxima – a seleção gulosa top-1 é ótima
-- (attention.py: ranking ordenado do melhor para o pior).
theorem roteamento_guloso_otimo {Agent : Type*} [DecidableEq Agent] (candidatos : Finset
  Agent)
  (utilidade : Agent → Real) (h : candidatos.Nonempty) :
    ∃ melhor ∈ candidatos, ∀ a ∈ candidatos, utilidade a ≤ utilidade melhor :=
  Finset.exists_max_image candidatos utilidade h

-- T4: subtrair constante (o máximo, na implementação) não muda os pesos –
-- estabilidade numérica sem mudar a decisão (attention.py: exps de s - m).
theorem softmax_invariante_max ι { : Type*} [DecidableEq ι] [Fintype ι] (f : ι → Real) [
  Nonempty ι]
  (c : Real) (i : ι) :
    Real.exp (f i - c) / ∑ j, Real.exp (f j - c)
    = Real.exp (f i) / ∑ j, Real.exp (f j) := by
  have he : Real.exp c ≠ 0 := ne_of_gt (Real.exp_pos c)

```

```

have hS :  $\sum (j, \text{Real.exp } (f j)) \neq 0 := \text{ne\_of\_gt}$ 
  (Finset.sum_pos (fun k _ => Real.exp_pos (f k)) Finset.univ_nonempty)
simp only [Real.exp_sub]
rw <[ Finset.sum_div]
field_simp

-- T5: cada peso está em (0,1) com ao menos dois candidatos – probabilidade genuína.
theorem softmax_mem_Ioo ι{ : Type*} [DecidableEq ι] [Fintype ι] (f : ι → Real) [Nonempty ι]
  [Nontrivial ι] (i : ι) :
  0 < Real.exp (f i) /  $\sum j, \text{Real.exp } (f j) \wedge \text{Real.exp } (f i) / \sum j, \text{Real.exp } (f j) < 1 :=
  by
  have hS : 0 <  $\sum j, \text{Real.exp } (f j) :=
    \text{Finset.sum\_pos } (\text{fun } k \_ => \text{Real.exp\_pos } (f k)) \text{Finset.univ\_nonempty}$ 
  refine (div_pos (Real.exp_pos _) hS, ?)
  rw [div_lt_one hS]
  obtain (j, )hji := exists_ne i
  have hR : 0 < Finset.sum (Finset.univ.erase i) (fun k => Real.exp (f k)) :=
    Finset.sum_pos (fun k _ => Real.exp_pos _) (j, Finset.mem_erase.mpr (hji, Finset.
      mem_univ ))j
  have hsplit := Finset.add_sum_erase Finset.univ (fun k => Real.exp (f k)) (Finset.mem_univ
    i)
  linarith

-- T6: limite inferior – fecha E2 bilateral (espelho de T2).
theorem combinacao_convexa_limitada_baixo ι{ : Type*} (s : Finset ι) (w x : ι → Real) (m :
  Real)
  (hsum : Finset.sum s (fun i => w i) = 1) (hnn :  $\forall i \in s, 0 \leq w i$ ) (hle :  $\forall i \in s, m \leq x$ 
  i) :
  m ≤ Finset.sum s (fun i => w i * x i) := by
  calc m = Finset.sum s (fun i => w i * m) := by rw <[ Finset.sum_mul, hsum, one_mul]
  _ ≤ Finset.sum s (fun i => w i * x i) :=
    Finset.sum_le_sum (fun i hi => mul_le_mul_of_nonneg_left (hle i hi) (hnn i hi))

-- T7: candidato único recebe peso 1 – complementa T5 (que exige Nontrivial).
-- Fecha a objeção do singleton: os dois casos estão cobertos.
theorem softmax_peso_total_singleton ι{ : Type*} [DecidableEq ι] [Fintype ι] (f : ι → Real)
  [Nonempty ι] (i : ι) (h :  $\forall j, j = i$ ) :
  Real.exp (f i) /  $\sum j, \text{Real.exp } (f j) = 1 := by
  have hS :  $\sum (j, \text{Real.exp } (f j)) = \text{Real.exp } (f i) :=
    \text{Finset.sum\_eq\_single\_of\_mem } i (\text{Finset.mem\_univ } i) (\text{fun } j \_ \text{hji} => \text{absurd } (h j) \text{hji})$ 
  rw [hS, div_self (ne_of_gt (Real.exp_pos _))]

-- T8: o peso cresce estritamente com o próprio escore (demais fixos) –
-- justifica a regra de decisão: maior escore, maior peso.
theorem peso_cresce_com_escore (C : Real) (hC : 0 < C) :
  StrictMono (fun x : Real => Real.exp x / (Real.exp x + C)) := by
  intro a b hab$$ 
```

```

have hea : 0 < Real.exp a := Real.exp_pos a
have heb : 0 < Real.exp b := Real.exp_pos b
rw [÷div_lt_div_iff (add_pos hea hC) (add_pos heb hC)]
have h1 : Real.exp a * C < Real.exp b * C :=
  mul_lt_mul_of_pos_right (Real.exp_strictMono hab) hC
linarith

-- T9: numa lista pairwise-decrescente, todo elemento dos take k domina todo
-- elemento do resto – o padrão top-k do ranking (sort + take).
theorem topk_dominam_resto (l : List Real) (k : ℕ) (hp : l.Pairwise (· ≥ ·))
  (x : Real) (hx : x ∈ l.drop k) (y : Real) (hy : y ∈ l.take k) : y ≥ x := by
  have happ : l.take k ++ l.drop k = l := List.take_append_drop k l
  rw ←[ happ] at hp
  rw [List.pairwise_append] at hp
  exact hp.2.2 y hy x hx

-- T10: esfriar a temperatura aumenta o peso do maximizador único – nitidez
-- (literatura de calibração por temperatura; aqui em forma elementar, sem limites).
theorem temperatura_aguca_maximo ι{ : Type*} [DecidableEq ι] [Fintype ι] (f : ι → Real)
  [Nonempty ι] [Nontrivial ι] (iest : ι) (hmax : ∀ j, j ≠ iest → f j < f iest)
  (τ₁ (τ₂ : Real) (τh1 : 0 < τ₁) (τh2 : 0 < τ₂) (hlt : τ₁ < τ₂) :
  Real.exp (f iest / τ₂) / ∑ j, Real.exp (f j / τ₂)
  < Real.exp (f iest / τ₁) / ∑ j, Real.exp (f j / τ₁) := by
  have hS1 : 0 < ∑ j, Real.exp (f j / τ₁) :=
    Finset.sum_pos (fun k _ => Real.exp_pos _) Finset.univ_nonempty
  have hS2 : 0 < ∑ j, Real.exp (f j / τ₂) :=
    Finset.sum_pos (fun k _ => Real.exp_pos _) Finset.univ_nonempty
  have hinv : 1 / τ₂ < 1 / τ₁ := one_div_lt_one_div_of_lt τh1 hlt
  have hlt_of_ne : ∀ j, j ≠ iest →
    Real.exp (f iest / τ₂) * Real.exp (f j / τ₁)
    < Real.exp (f iest / τ₁) * Real.exp (f j / τ₂) := by
    intro j hji
    have hfj : f j < f iest := hmax j hji
    have hdiff : (f iest / τ₁ + f j / τ₂) - (f iest / τ₂ + f j / τ₁)
      = (f iest - f j) * (1 / τ₁ - 1 / τ₂) := by
      field_simp
      ring
    have hpos : 0 < (f iest - f j) * (1 / τ₁ - 1 / τ₂) :=
      mul_pos (sub_pos.mpr hfj) (sub_pos.mpr hinv)
    have hlt' : f iest / τ₂ + f j / τ₁ < f iest / τ₁ + f j / τ₂ := by
      linarith
    calc Real.exp (f iest / τ₂) * Real.exp (f j / τ₁)
      = Real.exp (f iest / τ₂ + f j / τ₁) := (Real.exp_add _ _).symm
      < Real.exp (f iest / τ₁ + f j / τ₂) := Real.exp_strictMono hlt'
      = Real.exp (f iest / τ₁) * Real.exp (f j / τ₂) := Real.exp_add _ _
  have hterm : ∀ j ∈ Finset.univ,
    Real.exp (f iest / τ₂) * Real.exp (f j / τ₁)

```

```

      ≤ Real.exp (f iest / τ1) * Real.exp (f j / τ2) := by
intro j _
by_cases hji : j = iest
· subst hji; exact le_of_eq (mul_comm _ _)
· exact le_of_lt (hlt_of_ne j hji)
obtain ⟨j0, ⟩hj0 := exists_ne iest
have hstrict : Real.exp (f iest / τ2) * Real.exp (f j0 / τ1)
  < Real.exp (f iest / τ1) * Real.exp (f j0 / τ2) :=
  hlt_of_ne j0 hj0
have hsum : ∑( j, Real.exp (f iest / τ2) * Real.exp (f j / τ1))
  < ∑ j, Real.exp (f iest / τ1) * Real.exp (f j / τ2) :=
  Finset.sum_lt_sum hterm ⟨j0, Finset.mem_univ j0, ⟩hstrict
rw ←[ Finset.mul_sum, ← Finset.mul_sum] at hsum
rw [←div_lt_div_iff hS2 hS1]
linarith

-- T11: após subtrair majorante, expoentes ≤ 0 – sem overflow (truque do código).
theorem sem_overflow_apos_max ι{ : Type*} (f : ι → Real) (M : Real) (i : ι)
  (hM : ∀ j, f j ≤ M) : Real.exp (f i - M) ≤ 1 := by
  rw [Real.exp_le_one_iff]
  exact sub_nonpos.mpr (hM i)

```

APÊNDICE B – TRILHA DE REPRODUÇÃO

Lean 4 v4.34.1, Lake 5.0.0, elan 4.2.4; Mathlib rev. v4.34.1 (4760 oleans). Bancada principal: sementes 11, 22, 33, 44 e 55; 40 tarefas por semente. Ablações: sementes efetivas 10011, 10022, 10033, 10044 e 10055. Os códigos e transcrições reproduzidos são artefatos históricos; a execução integral da fonte atual não foi revalidada nesta revisão editorial. Comandos: `lakeenvleanRoteamentoAtencao.lean` (saída 0); `pytest 9` suítes (63 aprovados). Sincronia automatizada entre teoremas, testes, SPEC e axiomas: `scanners/diagnostic_pipeline` (SPEC-935-R767, 8 testes herméticos). A Tabela 11 reproduz os identificadores históricos dos repositórios e das licenças. Depósito público dos artefatos pendente do autor. OR-

Tabela 11 – Pinos verificados por `git ls-remote` (2026-10-09) e SHA-256 das licenças.

Repositório	Commit HEAD (8)	SHA-256 licença (8)
leanprover/lean4	5e96ee11	8b28515f
mathlib4	a02a63c2	b40930bb
batteries	b71aaf60	c71d239d
aesop	04879b1e	c71d239d
Z3Prover/z3	0d0e28c3	e617cad2
sympy/sympy	04a7da3a	07a5e981

Fonte: Elaboração própria (2026), a partir dos artefatos e das fontes identificados no texto.

CID: <https://orcid.org/0000-0001-8996-2887>. Contato: marceloclaro@gmail.com, <https://bit.ly/geomaker>.

APÊNDICE C – CÓDIGO DOS EXPERIMENTOS E DO PIPELINE

```
# -*- coding: utf-8 -*-
"""Bancada honesta do roteamento por atenção vs baselines (R756+).

Oráculo oculto (qualidade latente q): o roteador nunca vê q; vê apenas
cobertura, confiança ruidosa, carga e semântica. Comparação não-circular.
Determinístico por seed. Uso: python3 bancada_routing.py [--seeds 11,22,33,44,55]
"""

from __future__ import annotations

import argparse
import json
import math
import os
import random
import statistics
import sys
import time

sys.path.insert(0, "/home/marceloclaro/opencode-ecosystem-core")

from transformer.attention import AttentionRouter # noqa: E402

VOCAB = [f"cap_{c}" for c in
          ["formal", "revisao", "dados", "codigo", "geo", "bio", "nlp",
           "visao", "ml", "qa", "latex", "etica"]]
NOISE_WORDS = ["análise", "síntese", "relatório", "urgente", "rotina", "projeto"]

def make_pool(rng: random.Random, n_agents: int = 40):
    pool, latent = [], {}
    for k in range(n_agents):
        aid = f"ag_{k:02d}"
        caps = rng.sample(VOCAB, rng.randint(3, 6))
        q = rng.random()
        pool.append({
            "agent_id": aid,
            "status": "available",
            "capabilities": caps,
            "description": f"Agente {aid} especialista em " + " ".join(caps),
            "load": round(rng.uniform(0.0, 0.9), 3),
            "confidence_score": round(min(1.0, max(0.0, q + rng.gauss(0, 0.15)))), 3),
        })
        latent[aid] = q
    return pool, latent
```

```

def make_task(rng: random.Random, pool):
    for _ in range(50): # garante >=4 elegíveis
        req = sorted(rng.sample(VOCAB, rng.randint(2, 3)))
        elig = [c for c in pool if set(req) <= set(c["capabilities"])]
        if len(elig) >= 4:
            desc = ("Tarefa requer " + ", ".join(req) + " "
                    + " ".join(rng.sample(NOISE_WORDS, 2)))
            return desc, req, elig
    raise RuntimeError("falha ao gerar tarefa elegível")

def oracle_u(card, req, latent):
    cov = len(set(card["capabilities"] & set(req)) / len(req))
    return cov * (0.4 + 0.6 * latent[card["agent_id"]])

def ci95(vals):
    n = len(vals)
    m = statistics.fmean(vals)
    if n < 2:
        return m, 0.0
    sd = statistics.pstdev(vals)
    return m, 1.96 * sd / math.sqrt(n)

def run_seed(seed: int, n_tasks: int = 40):
    rng = random.Random(seed)
    router = AttentionRouter()
    pool, latent = make_pool(rng)
    out = []
    for t in range(n_tasks):
        desc, req, elig = make_task(rng, pool)
        oracle = {c["agent_id"]: oracle_u(c, req, latent) for c in elig}
        best = max(oracle.values())
        row = {"seed": seed, "task": t, "n_elig": len(elig),
              "oracle_best": best}
        # atenção
        t0 = time.perf_counter()
        ranking = router.route(desc, req, pool)
        dt = time.perf_counter() - t0
        top1 = ranking[0][0]
        row["atencao_escolha"] = top1
        row["atencao_regret"] = best - oracle[top1]
        row["atencao_hit"] = 1 if oracle[top1] == best else 0
        row["atencao_lat_ms"] = dt * 1000.0

```

```

row["softmax_sum"] = sum(w for _, w in ranking)
# baselines sobre elegíveis
row["aleatorio_escolha"] = rng.choice(elig)["agent_id"]
row["menor_carga_escolha"] = min(elig, key=lambda c: c["load"])["agent_id"]
covs = [(len(set(c["capabilities"]) & set(req)), c) for c in elig]
maxcov = max(c[0] for c in covs)
row["maior_cobertura_escolha"] = rng.choice(
    [c["agent_id"] for v, c in covs if v == maxcov])
for b in ("aleatorio", "menor_carga", "maior_cobertura"):
    pick = row[f"{b}_escolha"]
    row[f"{b}_regret"] = best - oracle[pick]
    row[f"{b}_hit"] = 1 if oracle[pick] == best else 0
out.append(row)
return out

```

```

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--seeds", default="11,22,33,44,55")
    ap.add_argument("--tasks", type=int, default=40)
    args = ap.parse_args()
    rows = []
    for s in [int(x) for x in args.seeds.split(",")]:
        rows += run_seed(s, args.tasks)
    res = {"seeds": args.seeds, "n_decisoos": len(rows)}
    for b in ("atencao", "aleatorio", "menor_carga", "maior_cobertura"):
        m, h = ci95([r[f"{b}_regret"] for r in rows])
        hm, hh = ci95([r[f"{b}_hit"] for r in rows])
        res[b] = {"regret_medio": m, "regret_ci95": h,
                  "hit_rate": hm, "hit_ci95": hh}
    lat = [r["atencao_lat_ms"] for r in rows]
    res["atencao_lat_ms"] = {"media": statistics.fmean(lat),
                             "max": max(lat)}
    dev = max(abs(r["softmax_sum"] - 1.0) for r in rows)
    res["softmax_desvio_max"] = dev
    res["amostra"] = rows[:3]
    base = os.path.dirname(os.path.abspath(__file__))
    with open(os.path.join(base, "resultado_bancada.json"), "w",
              encoding="utf-8") as f:
        json.dump(res, f, ensure_ascii=False, indent=2)
    for b in ("atencao", "aleatorio", "menor_carga", "maior_cobertura"):
        r = res[b]
        print(f"{b:15s} regret={r['regret_medio']:.4f}±{r['regret_ci95']:.4f} "
              f"hit={r['hit_rate']:.3f}±{r['hit_ci95']:.3f}")
    print(f"lat_med={res['atencao_lat_ms']['media']:.2f}ms "
          f"max={res['atencao_lat_ms']['max']:.2f}ms "
          f"softmax_dev_max={dev:.2e} n={len(rows)}")

```

```

if __name__ == "__main__":
    main()

# -*- coding: utf-8 -*-
"""Ablações da bancada: ataca a circularidade da confiança e compara forte.

- conf_embaralhada: permuta confidence_score (quebra proxy com q).
- conf_anticorrelacionada: conf = 1 - conf (inversão adversarial).
- conf_ruidosa: ruído N(0, 0.4) reamostrado (mal calibrada).
- media_igual: utilidade = média das 4 cabeças reais (comparador forte).
- sem_confianca: utilidade sem a cabeça confidence, renormalizada.
- Estatística pareada por decisão: diff de regret vs atenção + taxa de vitória.
Uso: python3 bancada_ablacao.py (lê mesmos seeds/tasks da bancada).
"""

from __future__ import annotations

import json
import math
import os
import random
import statistics
import sys

sys.path.insert(0, "/home/marceloclaro/opencode-ecosystem-core")

from transformer.attention import AttentionRouter # noqa: E402

from bancada_routing import ( # noqa: E402
    make_pool, make_task, ci95,
)

BASE = os.path.dirname(os.path.abspath(__file__))
PESOS = {"semantic": 0.30, "capability": 0.35, "confidence": 0.25, "load": 0.10}

def top1_ordenado(pesos):
    return sorted(pesos.items(), key=lambda kv: (-kv[1], kv[0]))[0][0]

def oracle_qw(card, req, latent, qw):
    cov = len(set(card["capabilities"] & set(req)) / len(req))
    return cov * ((1.0 - qw) + qw * latent[card["agent_id"]])

def run_seed(seed: int, n_tasks: int = 40, qw: float = 0.6):

```

```

rng = random.Random(10_000 + seed)
router = AttentionRouter()
pool, latent = make_pool(rng)
rows = []
for t in range(n_tasks):
    desc, req, elig = make_task(rng, pool)
    eids = [c["agent_id"] for c in elig]
    oracle = {c["agent_id"]: oracle_qw(c, req, latent, qw) for c in elig}
    best = max(oracle.values())
    ev = router._evaluate(desc, req, pool)
    heads, util = ev["heads"], ev["utility"]
    row = {"seed": seed, "task": t, "oracle_best": best}
    # atenção real
    pick = ev["ranking"][0][0]
    row["atencao"] = best - oracle[pick]
    # pools com confiança atacada
    confs = [c["confidence_score"] for c in pool]
    for nome, novo in (
        ("conf_embaralhada", rng.sample(confs, len(confs))),
        ("conf_anticorrelacionada",
         [round(1.0 - v, 3) for v in confs]),
        ("conf_ruidosa",
         [round(min(1.0, max(0.0, v + rng.gauss(0, 0.4)))]
          for v in confs)),
    ):
        pool2 = [dict(c, confidence_score=n)
                  for c, n in zip(pool, novo)]
        r2 = router.route(desc, req, pool2)
        row[nome] = best - oracle[r2[0][0]]
    # comparadores fortes sobre as cabeças reais
    util_eq = {a: sum(heads[h][a] for h in PESOS) / 4.0
               for a in eids}
    row["media_igual"] = best - oracle[top1_ordenado(util_eq)]
    wn = {h: PESOS[h] / 0.75 for h in PESOS if h != "confidence"}
    util_nc = {a: sum(wn[h] * heads[h][a] for h in wn)
               for a in eids}
    row["sem_confianca"] = best - oracle[top1_ordenado(util_nc)]
    rows.append(row)
return rows

```

```

def main():
    import argparse
    ap = argparse.ArgumentParser()
    ap.add_argument("--qw", type=float, default=0.6)
    args = ap.parse_args()
    rows = []

```

```

for s in (11, 22, 33, 44, 55):
    rows += run_seed(s, qw=args.qw)
n = len(rows)
res = {"n_decisoos": n, "qw": args.qw, "estrategias": {}}
base = [r["atencao"] for r in rows]
for k in ("atencao", "conf_embaralhada", "conf_anticorrelacionada",
          "conf_ruidosa", "media_igual", "sem_confianca"):
    m, h = ci95([r[k] for r in rows])
    res["estrategias"][k] = {"regret_medio": m, "regret_ci95": h}
for k in res["estrategias"]:
    if k == "atencao":
        continue
    diffs = [r[k] - r["atencao"] for r in rows]
    m, h = ci95(diffs)
    wins = sum(1 for r in rows if r["atencao"] <= r[k]) / n
    res["estrategias"][k]["diff_pareado_medio"] = m
    res["estrategias"][k]["diff_pareado_ci95"] = h
    res["estrategias"][k]["atencao_vence_ou_empata"] = wins
tag = f"_q{args.qw}".replace(".", "p")
with open(os.path.join(BASE, f"resultado_ablacao{tag}.json"), "w",
          encoding="utf-8") as f:
    json.dump(res, f, ensure_ascii=False, indent=2)
print(f"qw={args.qw}")
for k, v in res["estrategias"].items():
    extra = (f" diff={v['diff_pareado_medio']:.4f}±{v['diff_pareado_ci95']:.4f}"
             f" win={v['atencao_vence_ou_empata']:.3f}"
             if k != "atencao" else "")
    print(f"{k:24s} regret={v['regret_medio']:.4f}±{v['regret_ci95']:.4f}"
          f"{extra}")

if __name__ == "__main__":
    main()

# -*- coding: utf-8 -*-
"""Cálculos ainda não correlacionados (R763): Pearson entre variáveis que a
bancada principal não cruzou. Reexecuta os seeds (determinístico) coletando,
por decisão: regret, acerto, n_elegíveis, dispersão do oráculo, escores das
4 cabeças por elegível e qualidade latente. Sem pacotes externos.
Uso: python3 correlacoes.py
"""

from __future__ import annotations

import json
import math
import os
import random

```

```

import statistics
import sys

sys.path.insert(0, "/home/marceloclaro/opencode-ecosystem-core")

from transformer.attention import AttentionRouter # noqa: E402
from bancada_routing import ( # noqa: E402
    make_pool, make_task,
)
from bancada_ablacao import oracle_qw # noqa: E402

BASE = os.path.dirname(os.path.abspath(__file__))

def pearson(xs, ys):
    n = len(xs)
    mx, my = statistics.fmean(xs), statistics.fmean(ys)
    cov = sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / n
    sx = math.sqrt(sum((x - mx) ** 2 for x in xs) / n)
    sy = math.sqrt(sum((y - my) ** 2 for y in ys) / n)
    if sx == 0.0 or sy == 0.0:
        return None
    return cov / (sx * sy)

def main(qw: float = 0.6):
    router = AttentionRouter()
    regret, hit, elig, spread = [], [], [], []
    heads = {"semantic": [], "capability": [], "confidence": [], "load": []}
    conf_q = ([], [])
    for seed in (11, 22, 33, 44, 55):
        rng = random.Random(10_000 + seed)
        pool, latent = make_pool(rng)
        for _ in range(40):
            desc, req, elig = make_task(rng, pool)
            oracle = {c["agent_id"]: oracle_qw(c, req, latent, qw)
                      for c in elig}
            best = max(oracle.values())
            lo = min(oracle.values())
            ev = router._evaluate(desc, req, pool)
            pick = ev["ranking"][0][0]
            regret.append(best - oracle[pick])
            hit.append(1 if oracle[pick] == best else 0)
            elig.append(len(elig))
            spread.append(best - lo)
        for h in heads:
            for c in elig:

```

```

        heads[h].append(ev["heads"][h][c["agent_id"]])
    for c in elig:
        conf_q[0].append(c["confidence_score"])
        conf_q[1].append(latent[c["agent_id"]])
res = {
    "n_decisoos": len(regret),
    "nota": "null = variância zero (ex.: capability é 1.0 p/ todo elegível: o gate já
decidiu; a escolha corre nas outras 3 cabeças)",
    "pearson": {
        "regret_x_n_elegiveis": pearson(regret, nelig),
        "regret_x_dispersao_oraculo": pearson(regret, spread),
        "acerto_x_n_elegiveis": pearson(hit, nelig),
        "acerto_x_dispersao_oraculo": pearson(hit, spread),
        "confianca_x_q_latente": pearson(*conf_q),
    },
    "correlacao_cabecas": {},
}
nomes = list(heads)
for i in range(len(nomes)):
    for j in range(i + 1, len(nomes)):
        res["correlacao_cabecas"][f"{nomes[i]}_x_{nomes[j]}"] = \
            pearson(heads[nomes[i]], heads[nomes[j]])
with open(os.path.join(BASE, "correlacoes.json"), "w",
          encoding="utf-8") as f:
    json.dump(res, f, ensure_ascii=False, indent=2)
print(f"n={res['n_decisoos']}")
for k, v in res["pearson"].items():
    print(f"{k:28s} r={v:+.3f}" if v is not None else f"{k:28s} nula (constante)")
for k, v in res["correlacao_cabecas"].items():
    print(f"{k:28s} r={v:+.3f}" if v is not None else f"{k:28s} nula (constante)")

if __name__ == "__main__":
    main()

# -*- coding: utf-8 -*-
"""Contraprova numérica do _softmax real (float) vs teoria (Real).

Tenta REFUTAR, em 20k vetores aleatórios (magnitudes até  $\pm 1000$ , tamanhos
1..50): soma=1, pesos em [0,1], invariância à subtração do máximo,
preservação de argmax, singleton=[1.0], vaziao=[]. Qualquer violação acima
da tolerância é reportada e falha (exit 1).
"""
from __future__ import annotations

import random
import sys

```

```

sys.path.insert(0, "/home/marceloclaro/opencode-ecosystem-core")

from transformer.attention import _softmax # noqa: E402

TOL_SUM = 1e-9
TOL_SHIFT = 1e-12

def main(n: int = 20000, seed: int = 7):
    rng = random.Random(seed)
    viol = {"soma": 0, "limites": 0, "invariancia": 0, "argmax": 0,
           "singleton": 0, "vazio": 0}
    worst_sum = 0.0
    worst_shift = 0.0
    for k in range(n):
        m = rng.choice([1, 2, 3, 5, 8, 13, 25, 50])
        mag = rng.choice([1.0, 10.0, 100.0, 1000.0])
        xs = [rng.uniform(-mag, mag) for _ in range(m)]
        w = _softmax(xs)
        if len(w) != m:
            viol["soma"] += 1
            continue
        s = sum(w)
        worst_sum = max(worst_sum, abs(s - 1.0))
        if abs(s - 1.0) > TOL_SUM:
            viol["soma"] += 1
        if any(not (0.0 <= v <= 1.0) for v in w):
            viol["limites"] += 1
        if m == 1 and w != [1.0]:
            viol["singleton"] += 1
        mu = max(xs)
        w2 = _softmax([x - mu for x in xs])
        d = max(abs(a - b) for a, b in zip(w, w2))
        worst_shift = max(worst_shift, d)
        if d > TOL_SHIFT:
            viol["invariancia"] += 1
        amax_w = max(w)
        if max(xs[i] for i in range(m) if w[i] == amax_w) < max(xs):
            viol["argmax"] += 1
    if _softmax([]) != []:
        viol["vazio"] += 1
    total = sum(viol.values())
    print(f"vetores={n} violacoes={total} {viol}")
    print(f"pior_soma={worst_sum:.2e} pior_shift={worst_shift:.2e}")
    print("CONTRAPROVA_OK" if total == 0 else "CONTRAPROVA_FALHOU")
    return 1 if total else 0

```

```

if __name__ == "__main__":
    raise SystemExit(main())

# -*- coding: utf-8 -*-
"""Pipeline de Diagnóstico por Gaps – auditoria hermética entre artefatos (SPEC-935-R767).

Hermético: sem rede, subprocesso, LLM ou terceiros além da stdlib.
Todo resultado carrega `candidato_a_inspecao + exige_validacao_externa`.
Não substitui compilação, testes ou validação externa.
"""

from __future__ import annotations

import os
import re
from typing import Any, Dict, List

SPEC_ID = "SPEC-935-R767"
GERADOR = "marceloclaro"
TIPOS = ("nomes", "cobertura")

def _ler(arquivo: str) -> str | None:
    try:
        with open(arquivo, encoding="utf-8") as fh:
            return fh.read()
    except (OSError, UnicodeError):
        return None

def _nomes(texto: str, regex: str) -> List[str]:
    try:
        padrao = re.compile(regex)
    except re.error as exc:
        raise ValueError(f"regex inválida: {exc}")
    if padrao.groups < 1:
        raise ValueError("regex deve ter ao menos 1 grupo.")
    vistos: List[str] = []
    for m in padrao.finditer(texto):
        nome = m.group(1)
        if nome not in vistos:
            vistos.append(nome)
    return vistos

class DiagnosticPipeline:

```

```

"""Audita sincronia entre artefatos declarados em um plano."""

def varrer(self, plano: Dict[str, Any]) -> Dict[str, Any]:
    if not isinstance(plano, dict):
        raise ValueError("plano deve ser objeto.")
    conjuntos = plano.get("conjuntos")
    if not isinstance(conjuntos, list) or not conjuntos:
        raise ValueError("plano.conjuntos deve ser lista não vazia.")
    gaps: List[Dict[str, str]] = []
    total = 0
    for item in conjuntos:
        if not isinstance(item, dict):
            raise ValueError("cada checagem deve ser objeto.")
        tipo = item.get("tipo")
        cid = item.get("id") or "sem-id"
        if tipo not in TIPOS:
            raise ValueError(f"tipo inválido em '{cid}': {tipo!r}.")
        arquivo = item.get("arquivo")
        regex = item.get("regex")
        if not isinstance(arquivo, str) or not arquivo.strip():
            raise ValueError(f"arquivo ausente em '{cid}'.")
        if not isinstance(regex, str) or not regex.strip():
            raise ValueError(f"regex ausente em '{cid}'.")
        total += 1
        texto = _ler(arquivo)
        if texto is None:
            gaps.append({"id": cid, "motivo": "arquivo_ausente",
                        "detalhe": arquivo})
            continue
        if tipo == "nomes":
            self._checar_nomes(item, cid, texto, gaps)
        else:
            self._checar_cobertura(item, cid, texto, gaps)
    falharam = len([g["id"] for g in gaps])
    return {
        "spec_id": SPEC_ID,
        "gerador": GERADOR,
        "total_checks": total,
        "passaram": total - falharam,
        "falharam": falharam,
        "gaps": gaps,
        "rotulo": "candidato_a_inspecao",
        "exige_validacao_externa": True,
    }

def _checar_nomes(self, item: Dict[str, Any], cid: str,
                  texto: str, gaps: List[Dict[str, str]]) -> None:

```

```

esperados = item.get("esperados", [])
proibidos = item.get("proibidos", [])
if not isinstance(esperados, list) or not isinstance(proibidos, list):
    raise ValueError(f"esperados/proibidos devem ser listas em '{cid}'".)
achados = _nomes(texto, item["regex"])
for nome in esperados:
    if nome not in achados:
        gaps.append({"id": cid, "motivo": "faltante",
                     "detalhe": str(nome)})
for nome in achados:
    if nome not in [str(e) for e in esperados]:
        gaps.append({"id": cid, "motivo": "excedente",
                     "detalhe": str(nome)})
for padrao in proibidos:
    try:
        rx = re.compile(str(padrao))
    except re.error as exc:
        raise ValueError(f"regex proibida inválida em '{cid}': {exc}")
    if rx.search(texto):
        gaps.append({"id": cid, "motivo": "proibido",
                     "detalhe": str(padrao)})

def _checar_cobertura(self, item: Dict[str, Any], cid: str,
                      texto: str, gaps: List[Dict[str, str]]) -> None:
    ref = item.get("referencia")
    if not isinstance(ref, dict) or not isinstance(ref.get("arquivo"), str) \
        or not isinstance(ref.get("regex"), str):
        raise ValueError(f"referencia inválida em '{cid}'".)
    texto_ref = _ler(ref["arquivo"])
    if texto_ref is None:
        gaps.append({"id": cid, "motivo": "arquivo_ausente",
                     "detalhe": str(ref["arquivo"])})
    return
    for nome in _nomes(texto, item["regex"]):
        if nome not in _nomes(texto_ref, ref["regex"]):
            gaps.append({"id": cid, "motivo": "cobertura",
                         "detalhe": str(nome)})

# -*- coding: utf-8 -*-
"""Roteamento multicritério inspirado em atenção Transformer.

A implementação é uma heurística determinística e auditável, não uma camada
neural treinada. Capacidades obrigatórias e disponibilidade são *hard gates*;
somente os candidatos elegíveis recebem scores normalizados e pesos softmax.
"""

```

```
import math
```

```

from typing import Any, Dict, List, Optional, Tuple

from .embedder import TaskEmbedder

def _dot(a: List[float], b: List[float]) -> float:
    return math.fsum(x * y for x, y in zip(a, b))

def _unit_score(value: Any, default: float = 0.0) -> float:
    """Converte um valor numérico em score finito no intervalo ``[0, 1]``."""

    if isinstance(value, bool):
        return default
    try:
        score = float(value)
    except (TypeError, ValueError):
        return default
    if not math.isfinite(score):
        return default
    return max(0.0, min(1.0, score))

def _cosine_unit_interval(a: List[float], b: List[float]) -> float:
    """Mapeia similaridade cosseno de ``[-1, 1]`` para ``[0, 1]``."""

    norm_a = math.sqrt(_dot(a, a))
    norm_b = math.sqrt(_dot(b, b))
    if not math.isfinite(norm_a) or not math.isfinite(norm_b):
        return 0.5
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.5
    cosine = _dot(a, b) / (norm_a * norm_b)
    if not math.isfinite(cosine):
        return 0.5
    cosine = max(-1.0, min(1.0, cosine))
    return (cosine + 1.0) / 2.0

def _softmax(scores: List[float]) -> List[float]:
    if not scores:
        return []
    m = max(scores)
    exps = [math.exp(s - m) for s in scores]
    total = math.fsum(exps) or 1.0
    return [e / total for e in exps]

```

```

class AttentionRouter:
    """Ranqueia agentes elegíveis por quatro critérios normalizados.

    Melhoria R347 – Semantic Matching + Skill Handbook:
    - A cabeça 'semantic' usa agora o SemanticMatcher (embeddings semânticos
      via LiteRT-LM/Colibri com fallback hash) em vez de feature hashing puro.
    - A cabeça 'confidence' usa o SkillHandbook (confiança per-skill) quando
      disponível, em vez de confidence ledger genérico.
    - Fallback automático para o comportamento legado se SemanticMatcher
      não estiver disponível.
    """

    _HEAD_NAMES = ("semantic", "capability", "confidence", "load")
    HEAD_WEIGHTS = {
        "semantic": 0.30,
        "capability": 0.35,
        "confidence": 0.25,
        "load": 0.10,
    }

    def __init__(self):
        self._head_weights = self._validated_weights(self.HEAD_WEIGHTS)
        self.embedder = TaskEmbedder()

        # SemanticMatcher (Melhoria #2) – carregamento lazy
        self._semantic_matcher = None

    @property
    def semantic_matcher(self):
        if self._semantic_matcher is None:
            try:
                from transformer.semantic_matcher import semantic_matcher
                self._semantic_matcher = semantic_matcher
            except Exception:
                self._semantic_matcher = False # False = falhou
        return self._semantic_matcher if self._semantic_matcher else None

    @classmethod
    def _validated_weights(cls, weights: Dict[str, float]) -> Dict[str, float]:
        """Exige uma combinação convexa completa para evitar scores ambíguos."""

        if set(weights) != set(cls._HEAD_NAMES):
            raise ValueError("HEAD_WEIGHTS deve conter exatamente as quatro cabeças")
        normalized: Dict[str, float] = {}
        for name in cls._HEAD_NAMES:
            value = weights[name]

```

```

        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise ValueError(f"peso inválido para a cabeça {name!r}")
        score = float(value)
        if not math.isfinite(score) or not 0.0 <= score <= 1.0:
            raise ValueError(f"peso fora de [0, 1] para a cabeça {name!r}")
        normalized[name] = score
    if not math.isclose(math.fsum(normalized.values()), 1.0, abs_tol=1e-12):
        raise ValueError("HEAD_WEIGHTS deve somar 1")
    return normalized

# -----
# Cabeças de atenção
# -----
def _head_semantic(self, task_vec: List[float], cards: List[Dict],
                    description: str = "", required: Optional[List[str]] = None) -> List[
float]:
    """Similaridade semântica: SemanticMatcher > feature hashing legado.

    Se o SemanticMatcher estiver disponível, usa matching semântico
    por skills. Caso contrário, usa feature hashing (comportamento legado).
    """
    matcher = self.semantic_matcher

    # Tenta matching semântico por skills primeiro
    if matcher and description:
        try:
            # Busca no Handbook por skills que correspondem à descrição
            matches = matcher.match_capabilities(
                task_description=description,
                required_capabilities=required or [],
                top_k=20, # busca ampla
            )
            # Constrói score baseado nos matches
            match_scores = {}
            for m in matches:
                aid = m["agent_id"]
                if aid not in match_scores or m["score"] > match_scores[aid]:
                    match_scores[aid] = m["score"]

            result = []
            for card in cards:
                agent_id = card.get("agent_id", "")
                if agent_id in match_scores:
                    result.append(match_scores[agent_id])
                else:
                    # Agente sem match semântico – usa legado como fallback
                    result.append(

```

```

        _cosine_unit_interval(task_vec, self.embedder.embed_agent(card))
    )
    return result
except Exception:
    pass

# Fallback: feature hashing legado
return [
    _cosine_unit_interval(task_vec, self.embedder.embed_agent(card))
    for card in cards
]

def _head_capability(self, required: List[str], cards: List[Dict]) -> List[float]:
    """Cobertura direcionada das capacidades obrigatórias."""

    required_set = set(required)
    scores = []
    for card in cards:
        caps = set(card.get("capabilities", []))
        if not required_set:
            scores.append(1.0)
            continue
        scores.append(len(caps & required_set) / len(required_set))
    return scores

def _head_confidence(self, cards: List[Dict],
                    description: str = "") -> List[float]:
    """Confiança: SkillHandbook per-skill > confidence ledger genérico.

    Se o SemanticMatcher/SkillHandbook estiver disponível, usa a
    confiança média das skills do agente que correspondem à tarefa.
    Caso contrário, usa o confidence ledger (comportamento legado).
    """

    matcher = self.semantic_matcher

    # Tenta confidence per-skill primeiro
    if matcher and description:
        try:
            matches = matcher.match_capabilities(
                task_description=description,
                top_k=50,
            )
            # Constrói confidence score por agente baseado nas skills
            agent_confidences: Dict[str, List[float]] = {}
            for m in matches:
                aid = m["agent_id"]
                agent_confidences.setdefault(aid, []).append(m["confidence"])

```

```

        result = []
        for card in cards:
            agent_id = card.get("agent_id", "")
            if agent_id in agent_confidences:
                confs = agent_confidences[agent_id]
                result.append(sum(confs) / len(confs)) # média
            else:
                result.append(_unit_score(card.get("confidence_score", 0.5), 0.5))
        return result
    except Exception:
        pass

# Fallback: confidence ledger genérico
return [_unit_score(card.get("confidence_score", 0.5), 0.5) for card in cards]

def _head_load(self, cards: List[Dict]) -> List[float]:
    """Capacidade livre declarada; ausência de carga equivale a livre."""

    return [1.0 - _unit_score(card.get("load", 0.0), 1.0) for card in cards]

@staticmethod
def _hard_gate_reasons(required: List[str], card: Dict[str, Any]) -> List[str]:
    """Explica por que um cartão falhou nos gates não negociáveis."""

    reasons: List[str] = []
    if card.get("status") != "available":
        reasons.append("status_not_available")
    capabilities = card.get("capabilities", ())
    if isinstance(capabilities, str):
        capabilities = (capabilities,)
    try:
        capability_set = set(capabilities)
    except TypeError:
        capability_set = set()
    reasons.append("invalid_capabilities")
    missing = sorted(set(required) - capability_set)
    if missing:
        reasons.append("missing_capabilities:" + ", ".join(missing))
    return reasons

def _partition_cards(
    self,
    required: List[str],
    cards: List[Dict],
) -> Tuple[List[Dict], Dict[str, List[Dict]]]:
    eligible_by_id: Dict[str, List[Dict]] = {}

```

```

excluded: Dict[str, List[str]] = {}
seen: set[str] = set()
for index, card in enumerate(cards):
    if not isinstance(card, dict):
        excluded[f"candidate-{index}"] = ["invalid_card"]
        continue
    raw_agent_id = card.get("agent_id")
    if not isinstance(raw_agent_id, str) or not raw_agent_id.strip():
        excluded[f"candidate-{index}"] = ["missing_agent_id"]
        continue
    agent_id = raw_agent_id.strip()
    if agent_id in seen:
        eligible_by_id.pop(agent_id, None)
        excluded[agent_id] = ["duplicate_agent_id"]
        continue
    seen.add(agent_id)
    reasons = self._hard_gate_reasons(required, card)
    if reasons:
        excluded[agent_id] = reasons
    else:
        normalized = dict(card)
        normalized["agent_id"] = agent_id
        eligible_by_id[agent_id] = normalized
return list(eligible_by_id.values()), excluded

def _evaluate(
    self,
    description: str,
    required_capabilities: List[str],
    cards: List[Dict],
) -> Dict[str, Any]:
    """Executa gates, cabeças, utilidade convexa e ranking uma única vez."""

    if isinstance(required_capabilities, str):
        required = [required_capabilities]
    else:
        required = list(dict.fromkeys(required_capabilities))
    eligible, excluded = self._partition_cards(required, cards)
    agent_ids = [str(card["agent_id"]) for card in eligible]

    empty_heads = {name: {} for name in self._HEAD_NAMES}
    if not eligible:
        return {
            "eligible": [],
            "excluded": excluded,
            "heads": empty_heads,
            "utility": {},

```

```

        "weights": dict(self._head_weights),
        "ranking": [],
    }

    # 0 índice global de tarefas não pode alterar uma decisão idêntica.
    task_vec = self.embedder.embed_task(description, required, positional_index=0)
    scores_by_head = {
        "semantic": self._head_semantic(task_vec, eligible, description, required),
        "capability": self._head_capability(required, eligible),
        "confidence": self._head_confidence(eligible, description),
        "load": self._head_load(eligible),
    }
    heads = {
        name: dict(zip(agent_ids, scores_by_head[name]))
        for name in self._HEAD_NAMES
    }
    utility = {
        agent_id: math.fsum(
            self._head_weights[name] * heads[name][agent_id]
            for name in self._HEAD_NAMES
        )
        for agent_id in agent_ids
    }
    final_weights = dict(zip(agent_ids, _softmax([utility[item] for item in agent_ids])))
)

    ranking = sorted(
        final_weights.items(),
        key=lambda item: (-item[1], item[0]),
    )
    return {
        "eligible": [agent_id for agent_id, _ in ranking],
        "excluded": excluded,
        "heads": heads,
        "utility": utility,
        "weights": dict(self._head_weights),
        "ranking": ranking,
    }

# -----
# Atenção combinada
# -----
def route(self, description: str, required_capabilities: List[str],
          cards: List[Dict], positional_index: int = 0) -> List[Tuple[str, float]]:
    """
    Retorna ranking [(agent_id, attention_weight)] ordenado do melhor
    para o pior, com pesos softmax somando 1.
    """

```

```

        del positional_index # compatibilidade de API; deliberadamente não participa do
        score
        return self._evaluate(description, required_capabilities, cards)["ranking"]

def explain(self, description: str, required_capabilities: List[str],
            cards: List[Dict]) -> Dict[str, Any]:
    """Expõe gates, scores, utilidade, pesos e ranking da heurística."""

    explanation = self._evaluate(description, required_capabilities, cards)
    return {
        "task": description,
        "required_capabilities": list(required_capabilities),
        **explanation,
    }

# -*- coding: utf-8 -*-
"""
MCP Lean Proof Server – SPEC-935-R747
Envolve motores existentes (R444/R445) com contrato fail-closed e anti-overclaim.
Sem rede. Reutiliza lean4_verifier, autoformalizer e egraph quando disponível.
"""

from __future__ import annotations

import hashlib
import re
from typing import Any, Dict, List, Optional

from integrations.deepmind.lean4_verifier import Lean4ProofVerifier
from integrations.deepmind.autoformalizer import AutoFormalizerEngine

_VERIFIER = Lean4ProofVerifier()
_FORMALIZER = AutoFormalizerEngine(verifier=_VERIFIER)

_SORRY_RE = re.compile(r"\bsorry\b")
_ADMIT_RE = re.compile(r"\badmit\b")
_AXIOM_RE = re.compile(r"^\s*axiom\b", re.MULTILINE)

def _provenance() -> Dict[str, Any]:
    return {
        "has_compiler": bool(_VERIFIER.has_compiler),
        "lean_binary": _VERIFIER.lean_binary,
        "mathlib_version_hint": "Mathlib 4 (imports declarados no código)",
        "engine": "lean4_verifier R444 + autoformalizer R445 via R747",
    }

```

```

def lean_verify(code: str, project_dir: Optional[str] = None) -> Dict[str, Any]:
    """Verifica código Lean com fail-closed para sorry/admit/axiom não declarado. R754:
    project_dir usa lake-env."""
    if _SORRY_RE.search(code) or _ADMIT_RE.search(code):
        th = _VERIFIER.extract_theorem_name(code)
        tactics = _VERIFIER.extract_tactics(code)
        return {
            "is_valid": False,
            "status": "incomplete_sorry",
            "theorem_name": th,
            "tactics_used": tactics,
            "has_sorry": True,
            "errors": ["Bloqueio fail-closed: 'sorry' ou 'admit' detectado. Prova incompleta
            ."],
            "output_log": "bloqueado_sorry_admit",
            "lean_source": code,
            "provenance": {**_provenance(), "project_dir": project_dir, "via": "lake-env" if
            project_dir else "bare-lean"},
        }
    if _AXIOM_RE.search(code):
        th = _VERIFIER.extract_theorem_name(code)
        tactics = _VERIFIER.extract_tactics(code)
        return {
            "is_valid": False,
            "status": "axiom_nao_declarado",
            "theorem_name": th,
            "tactics_used": tactics,
            "has_sorry": False,
            "errors": ["Bloqueio fail-closed: 'axiom' exige declaração explícita e auditoria
            ."],
            "output_log": "bloqueado_axiom",
            "lean_source": code,
            "provenance": {**_provenance(), "project_dir": project_dir, "via": "lake-env" if
            project_dir else "bare-lean"},
        }
    res = _VERIFIER.verify_lean_code(code, project_dir=project_dir)
    d = res.to_dict()
    # Anti-overclaim: sem compilador real, nunca machine_checked.
    if not _VERIFIER.has_compiler and d.get("status") == "machine_checked":
        d["status"] = "syntax_verified"
        d["is_valid"] = True
        d["output_log"] = (d.get("output_log", "") + "\n[anti-overclaim R747: sem compilador
        , rebaixado para syntax_verified]").strip()
    d["provenance"] = {**_provenance(), "project_dir": project_dir, "via": "lake-env" if
    project_dir else "bare-lean"}
    return d

```

```

def lean_export(
    theorem_name: str,
    statement: str,
    tactics: List[str],
    imports: Optional[List[str]] = None,
    parameters: str = "",
) -> Dict[str, Any]:
    """Exporta teorema em Lean 4 com hash SHA-256 para custódia."""
    imports = imports or ["Mathlib.Tactic", "Mathlib.Data.Real.Basic"]
    src = _VERIFIER.format_theorem(
        theorem_name=theorem_name,
        statement=statement,
        proof_tactics=tactics,
        imports=imports,
        parameters=parameters,
    )
    return {
        "theorem_name": theorem_name,
        "lean_source": src,
        "sha256": hashlib.sha256(src.encode("utf-8")).hexdigest(),
        "imports": imports,
        "provenance": _provenance(),
    }

def lean_autoformalize(informal_text: str, domain: str = "algebra") -> Dict[str, Any]:
    """Autoformaliza texto informal para Lean 4 (R445) com chave lean_source."""
    out = _FORMALIZER.informal_to_lean4(informal_text=informal_text, domain=domain)
    lean_src = out.get("lean_code", "")
    out["lean_source"] = lean_src
    out["provenance"] = _provenance()
    return out

def lean_saturate(expr: str, rules: Optional[List[str]] = None) -> Dict[str, Any]:
    """Saturação E-Graph quando disponível; senão indisponível explícito."""
    try:
        from integrations.deepmind.egraph_rewriter import EqualitySaturationEngine
        eng = EqualitySaturationEngine()
        # API mínima: tenta simplificar se método existir, senão ecoa.
        simplify = getattr(eng, "simplify", None) or getattr(eng, "saturate", None)
        if callable(simplify):
            try:
                result = simplify(expr, rules or [])
            except TypeError:

```

```

        result = simplify(expr)
        return {"expr": expr, "result": str(result), "status": "saturado", "provenance":
        _provenance()}
        return {"expr": expr, "result": expr, "status": "motor_presente_sem_simplify", "
        provenance": _provenance()}
    except Exception as exc: # fail-closed explícito
        return {"expr": expr, "result": expr, "status": "indisponivel", "error": str(exc), "
        provenance": _provenance()}

# -*- coding: utf-8 -*-
"""
Testes TDD para SPEC-935-R747 – Núcleo Lean
Verificação total fail-closed, sem overclaim sem compilador.
"""

import hashlib
import json
import os
import re
import unittest

REPO = "/home/marceloclaro/opencode-ecosystem-core"
AGENTS = [
    "agents/catalog/49_agente_lean_verificacao_total.md",
    "agents/catalog/50_agente_lean_matematica_formal.md",
    "agents/catalog/51_agente_lean_tese_auditavel.md",
    "agents/catalog/52_agente_lean_software_critico.md",
    "agents/catalog/53_agente_lean_estudo_teoremas.md",
]
SKILLS = [
    ".opencode/skills/lean-verificacao-total/SKILL.md",
    ".opencode/skills/lean-tese-provas-auditaveis/SKILL.md",
]
PLUGIN_DIR = "plugins/lean-nucleo-verificacao"

class TestNucleoLeanR747(unittest.TestCase):

    def test_spec_r747_registered(self):
        from sdd.spec_engine import spec_registry
        spec = spec_registry.get("SPEC-935-R747")
        self.assertIsNotNone(spec, "SPEC-935-R747 deve estar registrada no SpecRegistry")

    def test_agentes_lean_existem(self):
        for rel in AGENTS:
            path = os.path.join(REPO, rel)
            self.assertTrue(os.path.isfile(path), f"agente ausente: {rel}")

```

```

        with open(path, encoding="utf-8") as f:
            content = f.read()
            self.assertIn("name:", content, f"frontmatter ausente em {rel}")
            self.assertTrue(
                ("fail-closed" in content.lower() or "fail_closed" in content.lower() or "
bloqueador" in content.lower()),
                f"agente {rel} deve declarar bloqueadores fail-closed",
            )

def test_skills_lean_existem(self):
    for rel in SKILLS:
        path = os.path.join(REPO, rel)
        self.assertTrue(os.path.isfile(path), f"skill ausente: {rel}")
        with open(path, encoding="utf-8") as f:
            content = f.read()
            self.assertIn("sorry", content.lower(), f"skill {rel} deve mencionar sorry")
            self.assertIn("admit", content.lower(), f"skill {rel} deve mencionar admit")

def test_mcp_lean_verify_failclosed(self):
    from integrations.lean_proof_mcp import lean_verify
    ruim = "import Mathlib.Tactic\ntheorem t (x : Nat) : x + 0 = x := by\n sorry"
    r1 = lean_verify(ruim)
    self.assertFalse(r1["is_valid"], "sorry deve invalidar")
    self.assertIn(r1["status"], {"incomplete_sorry", "axiom_nao_declarado", "
bloqueado_sorry_admit"})
    bom = "import Mathlib.Tactic\ntheorem t (x : Nat) : x + 0 = x := by\n simp"
    r2 = lean_verify(bom)
    self.assertTrue(r2["is_valid"])
    self.assertIn(r2["status"], {"syntax_verified", "machine_checked"})

def test_mcp_lean_export_hash(self):
    from integrations.lean_proof_mcp import lean_export
    out = lean_export("soma_zero", "(n : Nat) : n + 0 = n", ["simp"])
    self.assertIn("theorem soma_zero", out["lean_source"])
    self.assertRegex(out["sha256"], r"^[0-9a-f]{64}$")
    self.assertEqual(out["sha256"], hashlib.sha256(out["lean_source"].encode("utf-8")).
hexdigest())

def test_mcp_lean_autoformalize(self):
    from integrations.lean_proof_mcp import lean_autoformalize
    out = lean_autoformalize("para todo x natural, x + 0 = x", domain="algebra")
    src = out.get("lean_source", "")
    self.assertIn("theorem", src)
    self.assertIn("Mathlib", src)

def test_plugin_manifesto(self):
    base = os.path.join(REPO, PLUGIN_DIR)

```

```

self.assertTrue(os.path.isfile(os.path.join(base, "PLUGIN.md")))
manifest_path = os.path.join(base, "lean-nucleo.json")
self.assertTrue(os.path.isfile(manifest_path))
with open(manifest_path, encoding="utf-8") as f:
    man = json.load(f)
self.assertEqual(len(man.get("agentes", [])), 5)
self.assertEqual(len(man.get("skills", [])), 2)
self.assertIn("mcp", man)

def test_sem_overclaim_sem_compilador(self):
    from integrations.lean_proof_mcp import lean_verify
    from integrations.deepmind.lean4_verifier import Lean4ProofVerifier
    v = Lean4ProofVerifier()
    if not v.has_compiler:
        bom = "import Mathlib.Tactic\ntheorem t (x : Nat) : x + 0 = x := by\n  simp"
        r = lean_verify(bom)
        # Sem compilador real, nunca alegar machine_checked como prova total sem
        ressalva:
        # implementação deve retornar syntax_verified e has_compiler=False na proveniê
        ncia.
        self.assertNotEqual(r["status"], "machine_checked",
                            "sem compilador não deve alegar machine_checked")
        self.assertFalse(r["provenance"]["has_compiler"])

if __name__ == "__main__":
    unittest.main()

```

APÊNDICE D – RESULTADOS EM JSON

```
{
  "seeds": "11,22,33,44,55",
  "n_decisoos": 200,
  "atencao": {
    "regret_medio": 0.042129070196756734,
    "regret_ci95": 0.010485649589724554,
    "hit_rate": 0.625,
    "hit_ci95": 0.06709601329438285
  },
  "aleatorio": {
    "regret_medio": 0.1882442013158154,
    "regret_ci95": 0.021944298385093076,
    "hit_rate": 0.2,
    "hit_ci95": 0.055437171645025325
  },
  "menor_carga": {
    "regret_medio": 0.21672938169919892,
    "regret_ci95": 0.024984665750796456,
    "hit_rate": 0.215,
    "hit_ci95": 0.05693707228159874
  },
  "maior_cobertura": {
    "regret_medio": 0.2117704733933451,
    "regret_ci95": 0.021470227249410667,
    "hit_rate": 0.165,
    "hit_ci95": 0.051442999523744724
  },
  "atencao_lat_ms": {
    "media": 2.1652001788606867,
    "max": 31.241488992236555
  },
  "softmax_desvio_max": 1.1102230246251565e-16,
  "amostra": [
    {
      "seed": 11,
      "task": 0,
      "n_elig": 4,
      "oracle_best": 0.8823401876326291,
      "atencao_escolha": "ag_00",
      "atencao_regret": 0.0,
      "atencao_hit": 1,
      "atencao_lat_ms": 31.241488992236555,
      "softmax_sum": 1.0,
      "aleatorio_escolha": "ag_07",
    }
  ]
}
```

```

    "menor_carga_escolha": "ag_07",
    "maior_cobertura_escolha": "ag_07",
    "aleatorio_regret": 0.07739468623161194,
    "aleatorio_hit": 0,
    "menor_carga_regret": 0.07739468623161194,
    "menor_carga_hit": 0,
    "maior_cobertura_regret": 0.07739468623161194,
    "maior_cobertura_hit": 0
  },
  {
    "seed": 11,
    "task": 1,
    "n_elig": 6,
    "oracle_best": 0.9893160524792669,
    "atencao_escolha": "ag_32",
    "atencao_regret": 0.1860276732480003,
    "atencao_hit": 0,
    "atencao_lat_ms": 2.270071068778634,
    "softmax_sum": 1.0,
    "aleatorio_escolha": "ag_25",
    "menor_carga_escolha": "ag_03",
    "maior_cobertura_escolha": "ag_27",
    "aleatorio_regret": 0.5818477472094881,
    "aleatorio_hit": 0,
    "menor_carga_regret": 0.412825110091508,
    "menor_carga_hit": 0,
    "maior_cobertura_regret": 0.22854489374948017,
    "maior_cobertura_hit": 0
  },
  {
    "seed": 11,
    "task": 2,
    "n_elig": 4,
    "oracle_best": 0.8823401876326291,
    "atencao_escolha": "ag_00",
    "atencao_regret": 0.0,
    "atencao_hit": 1,
    "atencao_lat_ms": 1.2811570195481181,
    "softmax_sum": 1.0,
    "aleatorio_escolha": "ag_09",
    "menor_carga_escolha": "ag_03",
    "maior_cobertura_escolha": "ag_03",
    "aleatorio_regret": 0.177020923739184,
    "aleatorio_hit": 0,
    "menor_carga_regret": 0.3058492452448702,
    "menor_carga_hit": 0,
    "maior_cobertura_regret": 0.3058492452448702,

```

```

        "maior_cobertura_hit": 0
    }
]
}

{
    "n_decisoos": 200,
    "qw": 0.3,
    "estrategias": {
        "atencao": {
            "regret_medio": 0.014612694537431255,
            "regret_ci95": 0.0036154726699519023
        },
        "conf_embaralhada": {
            "regret_medio": 0.11213339580065519,
            "regret_ci95": 0.011432166803568037,
            "diff_pareado_medio": 0.09752070126322394,
            "diff_pareado_ci95": 0.012494013366794746,
            "atencao_vence_ou_empata": 0.92
        },
        "conf_anticorrelacionada": {
            "regret_medio": 0.18019856944840154,
            "regret_ci95": 0.008085414074453907,
            "diff_pareado_medio": 0.16558587491097027,
            "diff_pareado_ci95": 0.009703935661874915,
            "atencao_vence_ou_empata": 0.97
        },
        "conf_ruidosa": {
            "regret_medio": 0.07574354813780261,
            "regret_ci95": 0.010313987168316368,
            "diff_pareado_medio": 0.06113085360037135,
            "diff_pareado_ci95": 0.010899847177161647,
            "atencao_vence_ou_empata": 0.93
        },
        "media_igual": {
            "regret_medio": 0.03189929397291829,
            "regret_ci95": 0.006224721392250171,
            "diff_pareado_medio": 0.017286599435487035,
            "diff_pareado_ci95": 0.005592154404453648,
            "atencao_vence_ou_empata": 0.99
        },
        "sem_confianca": {
            "regret_medio": 0.10721549794753951,
            "regret_ci95": 0.011419866228391513,
            "diff_pareado_medio": 0.09260280341010826,
            "diff_pareado_ci95": 0.011837640901013566,
            "atencao_vence_ou_empata": 0.98
        }
    }
}

```

```

    }
  }
}

{
  "n_decisoos": 200,
  "qw": 0.6,
  "estrategias": {
    "atencao": {
      "regret_medio": 0.029225389074862506,
      "regret_ci95": 0.007230945339903801
    },
    "conf_embaralhada": {
      "regret_medio": 0.22426679160131038,
      "regret_ci95": 0.022864333607136075,
      "diff_pareado_medio": 0.1950414025264479,
      "diff_pareado_ci95": 0.02498802673358949,
      "atencao_vence_ou_empata": 0.92
    },
    "conf_anticorrelacionada": {
      "regret_medio": 0.3603971388968031,
      "regret_ci95": 0.016170828148907815,
      "diff_pareado_medio": 0.3311717498219406,
      "diff_pareado_ci95": 0.01940787132374983,
      "atencao_vence_ou_empata": 0.97
    },
    "conf_ruidosa": {
      "regret_medio": 0.15148709627560522,
      "regret_ci95": 0.020627974336632736,
      "diff_pareado_medio": 0.12226170720074273,
      "diff_pareado_ci95": 0.021799694354323294,
      "atencao_vence_ou_empata": 0.93
    },
    "media_igual": {
      "regret_medio": 0.06379858794583657,
      "regret_ci95": 0.012449442784500341,
      "diff_pareado_medio": 0.03457319887097407,
      "diff_pareado_ci95": 0.0111843088089073,
      "atencao_vence_ou_empata": 0.99
    },
    "sem_confianca": {
      "regret_medio": 0.21443099589507905,
      "regret_ci95": 0.022839732456783026,
      "diff_pareado_medio": 0.18520560682021653,
      "diff_pareado_ci95": 0.02367528180202713,
      "atencao_vence_ou_empata": 0.98
    }
  }
}

```

```

    }
  }

  {
    "n_decisoos": 200,
    "qw": 0.9,
    "estrategias": {
      "atencao": {
        "regret_medio": 0.043838083612293764,
        "regret_ci95": 0.010846418009855705
      },
      "conf_embaralhada": {
        "regret_medio": 0.3364001874019656,
        "regret_ci95": 0.03429650041070412,
        "diff_pareado_medio": 0.29256210378967185,
        "diff_pareado_ci95": 0.03748204010038424,
        "atencao_vence_ou_empata": 0.92
      },
      "conf_anticorrelacionada": {
        "regret_medio": 0.5405957083452047,
        "regret_ci95": 0.024256242223361722,
        "diff_pareado_medio": 0.49675762473291085,
        "diff_pareado_ci95": 0.029111806985624742,
        "atencao_vence_ou_empata": 0.97
      },
      "conf_ruidosa": {
        "regret_medio": 0.22723064441340782,
        "regret_ci95": 0.0309419615049491,
        "diff_pareado_medio": 0.18339256080111407,
        "diff_pareado_ci95": 0.03269954153148494,
        "atencao_vence_ou_empata": 0.93
      },
      "media_igual": {
        "regret_medio": 0.09569788191875485,
        "regret_ci95": 0.01867416417675051,
        "diff_pareado_medio": 0.0518597983064611,
        "diff_pareado_ci95": 0.016776463213360942,
        "atencao_vence_ou_empata": 0.99
      },
      "sem_confianca": {
        "regret_medio": 0.32164649384261856,
        "regret_ci95": 0.03425959868517454,
        "diff_pareado_medio": 0.2778084102303248,
        "diff_pareado_ci95": 0.035512922703040704,
        "atencao_vence_ou_empata": 0.98
      }
    }
  }
}

```

```

}

{
  "n_decisoos": 200,
  "nota": "null = variância zero (ex.: capability é 1.0 p/ todo elegível: o gate já decidiu;
    a escolha corre nas outras 3 cabeças)",
  "pearson": {
    "regret_x_n_elegiveis": 0.08078886940444673,
    "regret_x_dispersao_oraculo": -0.16105052985737997,
    "acerto_x_n_elegiveis": -0.19003708112748213,
    "acerto_x_dispersao_oraculo": 0.1323750519688058,
    "confianca_x_q_latente": 0.8538477569277693
  },
  "correlacao_cabecas": {
    "semantic_x_capability": null,
    "semantic_x_confidence": 0.05129095210259732,
    "semantic_x_load": 0.020376387718067296,
    "capability_x_confidence": null,
    "capability_x_load": null,
    "confidence_x_load": 0.0043553481183595385
  }
}

{
  "11": {
    "atencao_regret": 0.0752,
    "aleatorio_regret": 0.1889,
    "menor_carga_regret": 0.3157,
    "maior_cobertura_regret": 0.2319,
    "hit_atencao": 0.45,
    "n": 40
  },
  "22": {
    "atencao_regret": 0.0172,
    "aleatorio_regret": 0.2463,
    "menor_carga_regret": 0.2276,
    "maior_cobertura_regret": 0.231,
    "hit_atencao": 0.775,
    "n": 40
  },
  "33": {
    "atencao_regret": 0.0337,
    "aleatorio_regret": 0.1719,
    "menor_carga_regret": 0.2447,
    "maior_cobertura_regret": 0.2033,
    "hit_atencao": 0.55,
    "n": 40
  },

```

```
"44": {
  "atencao_regret": 0.0716,
  "aleatorio_regret": 0.1992,
  "menor_carga_regret": 0.2038,
  "maior_cobertura_regret": 0.2309,
  "hit_atencao": 0.6,
  "n": 40
},
"55": {
  "atencao_regret": 0.0129,
  "aleatorio_regret": 0.135,
  "menor_carga_regret": 0.0918,
  "maior_cobertura_regret": 0.1618,
  "hit_atencao": 0.75,
  "n": 40
}
}
```

APÊNDICE E – TRANSCRIÇÃO DA SUÍTE DE TESTES

```

tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_agentes_lean_existem
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_mcp_lean_autoformalize
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_mcp_lean_export_hash
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_mcp_lean_verify_failclosed
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_plugin_manifesto
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_sem_overclaim_sem_compilador
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_skills_lean_existem
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_spec_r747_registered
tests/test_r754_lake_env.py::TestLakeEnvR754::test_machine_checked_mathlib_no_projeto
tests/test_r754_lake_env.py::TestLakeEnvR754::test_prova_errada_falha_no_projeto
tests/test_r754_lake_env.py::TestLakeEnvR754::test_sorry_bloqueado_no_projeto
tests/test_r754_lake_env.py::TestLakeEnvR754::test_spec_r754_registered
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::
    test_arquivo_lean_existe_sem_sorry
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::test_axiomas_auditaveis
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::
    test_machine_checked_no_projeto
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::test_spec_r755_registered
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_doctor_check_17_lean4_egraph
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_egraph_add_and_union
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_egraph_congruence_closure
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_egraph_equality_saturation_simplification
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_egraph_sub_self_simplification
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_lean4_detect_sorry
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_lean4_format_theorem
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_lean4_syntax_delimiter_error
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_lean4_verify_valid_code
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_orchestrator_lean4_and_egraph_integration
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_spec_r444_registered
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_autoformalizer_informal_to_lean4
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_autoformalizer_lean4_to_informal

```

```

tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_cross_validation_aligned
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_cross_validation_detect_sorry
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_doctor_check_geometry
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_geometry_deductive_database
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_opencode_alphageometry_solve
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_orchestrator_geometry_and_autoformalize
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_spec_r445_registered
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_tikz_and_svg_renderer
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_wu_method_midpoint_polynomial_reduction
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_wu_method_pythagorean_reduction
tests/test_r711_pesquisador_polimata_github.py::test_allowlist_fechada_e_schema
tests/test_r711_pesquisador_polimata_github.py::test_validar_url_fail_closed
tests/test_r711_pesquisador_polimata_github.py::test_gerar_manifesto_schema_em_tmp
tests/test_r711_pesquisador_polimata_github.py::test_verificar_clones_fixtures_sinteticas
tests/test_r711_pesquisador_polimata_github.py::test_rotulo_epistemico_nunca_aprova
tests/test_r711_pesquisador_polimata_github.py::test_modulo_nao_executa_rede_nem_subprocessso
tests/test_r713_pinagem_federacao.py::test_pinar_federavel_fresco
tests/test_r713_pinagem_federacao.py::test_pinar_bloqueios
tests/test_r713_pinagem_federacao.py::test_fora_allowlist_sem_rede
tests/test_r713_pinagem_federacao.py::test_revalidar_tolera_falha_isolada
tests/test_r713_pinagem_federacao.py::test_emitir_pins_schema_tmp
tests/test_r713_pinagem_federacao.py::test_sem_rede_real_no_modulo
tests/test_r715_federacao_classes.py::test_classificar_21_e_rejeita_fora
tests/test_r715_federacao_classes.py::test_estavel_antigo_proposto_com_override
tests/test_r715_federacao_classes.py::test_ativo_velho_retido_e_sem_override_retido
tests/test_r715_federacao_classes.py::test_emitir_proposta_schema
tests/test_r767_diagnostic_pipeline.py::test_spec_r767_registered
tests/test_r767_diagnostic_pipeline.py::test_consistente_sem_gaps
tests/test_r767_diagnostic_pipeline.py::test_nome_faltante
tests/test_r767_diagnostic_pipeline.py::test_token_proibido
tests/test_r767_diagnostic_pipeline.py::test_cobertura_faltante
tests/test_r767_diagnostic_pipeline.py::test_plano_malformado
tests/test_r767_diagnostic_pipeline.py::test_arquivo_ausente_vira_gap
tests/test_r767_diagnostic_pipeline.py::test_gap_real_no_repo

```

63 tests collected in 9.17s

```

===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/marceloclaro/opencode-ecosystem-core
configfile: pytest.ini
plugins: anyio-4.14.1, typeguard-4.4.4
collecting ... collected 63 items

tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_agentes_lean_existem PASSED [ 1%]
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_mcp_lean_autoformalize PASSED [
  3%]
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_mcp_lean_export_hash PASSED [ 4%]
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_mcp_lean_verify_failclosed PASSED [
  6%]
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_plugin_manifesto PASSED [ 7%]
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_sem_overclaim_sem_compilador PASSED
  [ 9%]
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_skills_lean_existem PASSED [ 11%]
tests/test_r747_nucleo_lean.py::TestNucleoLeanR747::test_spec_r747_registered PASSED [ 12%]
tests/test_r754_lake_env.py::TestLakeEnvR754::test_machine_checked_mathlib_no_projeto PASSED
  [ 14%]
tests/test_r754_lake_env.py::TestLakeEnvR754::test_prova_errada_falha_no_projeto PASSED [
  15%]
tests/test_r754_lake_env.py::TestLakeEnvR754::test_sorry_bloqueado_no_projeto PASSED [ 17%]
tests/test_r754_lake_env.py::TestLakeEnvR754::test_spec_r754_registered PASSED [ 19%]
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::
  test_arquivo_lean_existe_sem_sorry PASSED [ 20%]
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::test_axiomas_auditaveis
  PASSED [ 22%]
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::
  test_machine_checked_no_projeto PASSED [ 23%]
tests/test_r755_teorema_roteamento.py::TestTeoremaRoteamentoR755::test_spec_r755_registered
  PASSED [ 25%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
  test_doctor_check_17_lean4_egraph PASSED [ 26%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
  test_egraph_add_and_union PASSED [ 28%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
  test_egraph_congruence_closure PASSED [ 30%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
  test_egraph_equality_saturation_simplification PASSED [ 31%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
  test_egraph_sub_self_simplification PASSED [ 33%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
  test_lean4_detect_sorry PASSED [ 34%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
  test_lean4_format_theorem PASSED [ 36%]

```

```

tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_lean4_syntax_delimiter_error PASSED [ 38%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_lean4_verify_valid_code PASSED [ 39%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_orchestrator_lean4_and_egraph_integration PASSED [ 41%]
tests/test_r444_lean4_egraph_saturation.py::TestLean4EGraphSaturationR444::
    test_spec_r444_registered PASSED [ 42%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_autoformalizer_informal_to_lean4 PASSED [ 44%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_autoformalizer_lean4_to_informal PASSED [ 46%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_cross_validation_aligned PASSED [ 47%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_cross_validation_detect_sorry PASSED [ 49%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_doctor_check_geometry PASSED [ 50%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_geometry_deductive_database PASSED [ 52%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_opencode_alphageometry_solve PASSED [ 53%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_orchestrator_geometry_and_autoformalize PASSED [ 55%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_spec_r445_registered PASSED [ 57%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_tikz_and_svg_renderer PASSED [ 58%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_wu_method_midpoint_polynomial_reduction PASSED [ 60%]
tests/test_r445_alphageometry_autoformalization.py::TestAlphaGeometryAutoformalizationR445::
    test_wu_method_pythagorean_reduction PASSED [ 61%]
tests/test_r711_pesquisador_polimata_github.py::test_allowlist_fechada_e_schema PASSED [
    63%]
tests/test_r711_pesquisador_polimata_github.py::test_validar_url_fail_closed PASSED [ 65%]
tests/test_r711_pesquisador_polimata_github.py::test gerar_manifesto_schema_em_tmp PASSED [
    66%]
tests/test_r711_pesquisador_polimata_github.py::test_verificar_clones_fixtures_sinteticas
    PASSED [ 68%]
tests/test_r711_pesquisador_polimata_github.py::test_rotulo_epistemico_nunca_aprova PASSED [
    69%]
tests/test_r711_pesquisador_polimata_github.py::test_modulo_nao_executa_rede_nem_subprocessso
    PASSED [ 71%]
tests/test_r713_pinagem_federacao.py::test_pinar_federavel_fresco PASSED [ 73%]
tests/test_r713_pinagem_federacao.py::test_pinar_bloqueios PASSED [ 74%]
tests/test_r713_pinagem_federacao.py::test_fora_allowlist_sem_rede PASSED [ 76%]
tests/test_r713_pinagem_federacao.py::test_revalidar_tolera_falha_isolada PASSED [ 77%]

```

```

tests/test_r713_pinagem_federacao.py::test_emitir_pins_schema_tmp PASSED [ 79%]
tests/test_r713_pinagem_federacao.py::test_sem_rede_real_no_modulo PASSED [ 80%]
tests/test_r715_federacao_classes.py::test_classificar_21_e_rejeita_fora PASSED [ 82%]
tests/test_r715_federacao_classes.py::test_estavel_antigo_proposto_com_override PASSED [
    84%]
tests/test_r715_federacao_classes.py::test_ativo_velho_retido_e_sem_override_retido PASSED [
    85%]
tests/test_r715_federacao_classes.py::test_emitir_proposta_schema PASSED [ 87%]
tests/test_r767_diagnostic_pipeline.py::test_spec_r767_registered PASSED [ 88%]
tests/test_r767_diagnostic_pipeline.py::test_consistente_sem_gaps PASSED [ 90%]
tests/test_r767_diagnostic_pipeline.py::test_nome_faltante PASSED [ 92%]
tests/test_r767_diagnostic_pipeline.py::test_token_proibido PASSED [ 93%]
tests/test_r767_diagnostic_pipeline.py::test_cobertura_faltante PASSED [ 95%]
tests/test_r767_diagnostic_pipeline.py::test_plano_malformado PASSED [ 96%]
tests/test_r767_diagnostic_pipeline.py::test_arquivo_ausente_vira_gap PASSED [ 98%]
tests/test_r767_diagnostic_pipeline.py::test_gap_real_no_repo PASSED [100%]

```

```

===== 63 passed in 248.00s (0:04:08) =====

```

```

== toolchain ==

```

```

Lean (version 4.34.1, x86_64-unknown-linux-gnu, commit 5045
    d0056413266e57c625dcd7c365b10e377c52, Release)

```

```

Lake version 5.0.0-src+5045d00 (Lean version 4.34.1)

```

```

elan 4.2.4 (227caca13 2026-08-25)

```

```

== python ==

```

```

Python 3.14.4

```

```

== pacotes ==

```

```

numpy                2.5.1
pytest               9.1.1
sympy                1.14.0
z3-solver            4.16.0.0

```

```

== maquina ==

```

```

8

```

	total	used	free	shared	buff/cache	available
Mem:	11	3	6	0	1	8

APÊNDICE F – HISTÓRICO DE AUDITORIA

Ciclos desta linhagem (registro compartilhado; homônimos legados com escores $\approx 0,9$ pertencem a outra linhagem e foram excluídos pelo filtro de escore e tema): A Tabela 12 apresenta os ciclos do registro evolutivo, cujos escores são internos.

Tabela 12 – Ciclos de evolução da linhagem Lean-roteamento (registros históricos).

Ciclo	Escore	Objeto registrado
R747	8.5	Núcleo Lean R747: 5 agentes, 2 skills, MCP lean_proof_mcp e plugin verificação total
R748	7.5	Sequência polimata Lean: manifesto + auditoria clones + orientação pinagem viva Z3/SymPy
R749	8.0	Autorização e validação R749: dossiê extensão R711 Lean sem federar
R750	8.2	Validação viva somente-leitura R750: API+LICENSE dos 4 Lean
R751	8.8	Execução R751: ls-remote 6 commits + sha256 6 LICENSE + pins+propor
R752	9.0	Aprovação R752: extensão allowlist 21→25 Lean + proposta 6/0
R753	8.7	Install/build R753: elan+stable v4.34.1 e fallback Mathlib-ausente honesto
R754	9.2	R754: lake new math + cache Mathlib + lake-env machine_checked + fallback tática-Mathlib
R755	9.3	R755: teorema eficiência roteamento-atenção 3/3 machine_checked
R756	9.4	R756: pacote 1+2+3 — invariância, Ioo, bilateral, 6/6 machine_checked
R757	8.5	R757: bancada interna + 2 revisões independentes + relatório honesto
R758	9.5	R758: máxima eficiência — T7+axiomas, fuzz 20k, ablações qw, sensibilidade
R759	9.0	Artigo PUCRS: revalidação exaustiva + PDF 5p compilado
R760	9.0	Artigo PUCRS ampliado: autoria+teoria+impactos, claim limpo, PDF 6p
R761	9.4	R761 T8 monotonicidade mais T9 top-k mais artigo 9 lemas
R762	9.1	Dissertacao ABNT 35p: 7 capitulos + QKV + 9 teoremas + estatistica total + Lean integral
R763	9.4	R763: T10 temperatura + T11 limite da exponencial deslocada + correlacoes Pearson + benchmarks externos mapeados
R764	9.3	R764: correcao minuciosa total + sincronia 11 + recalculos
R765	9.2	R765: dissertacao modular 9 modulos + 12 refs + teoria ampliada
R766	8.8	R766: revisao por gaps — pipeline inexistente + varredura total sem derivas
R767	9.2	R767: diagnostic_pipeline + gap real pego em estreia
R768	9.3	R768: pesquisa refeita — bateria fresca + docs 63-9 + PDFs
R769	8.9	R769: apendice didatico leigo + maswos_pipeline inexistente
R770	9.2	R770: modulos expandidos — enunciados integrais + pareada + ABNT vigente
R771	9.0	R771: versao arXiv — 11 lemas com provas + contexto didatico
R772	8.9	R772: preprint integralmente em PT-BR
R773	9.4	R773: dissertacao 80 laudas — 82p modular ABNT vigente

Fonte: Elaboração própria (2026), a partir do registro evolutivo local. Os escores históricos são autoavaliações internas e não equivalem a validação externa.

APÊNDICE G – ESPECIFICAÇÕES FORMAIS

```

---
spec_id: SPEC-935-R755
title: Teorema de Eficiência do Roteamento por Atenção (Análogo Transformer)
component: transformer/attention, plugins/lean-nucleo-verificacao/teoremas, tests/test_r755
status: proposed
round_id: R755
test_file: tests/test_r755_teorema_roteamento.py
---

# SPEC-935-R755 – Teorema de Eficiência do Roteamento por Atenção

## 1. Fundamento no código real

`transformer/attention.py`: `AttentionRouter` com 4 cabeças (`semantic 0.30`,
`capability 0.35`, `confidence 0.25`, `load 0.10`), `_validated_weights` exigindo
combinação convexa (soma 1, cada peso em `[0,1]`), `_softmax` com subtração do máximo
(estabilidade, como no Transformer) e ranking ordenado do melhor ao pior com pesos
somando 1. Hard gates antes de qualquer score; heurística determinística auditável,
não rede neural treinada.

## 2. Definição honesta de eficiência (sem overclaim)

"Mais eficiente" aqui significa **ótimo relativo à utilidade definida**, em 3 sentidos
precisos – nunca superioridade universal sobre outros orquestradores:

- **E1 calibração:** pesos de atenção formam distribuição de probabilidade (somam 1,
cada um em `(0,1)`); escores finais ficam limitados pelos componentes.
- **E2 limitação convexa:** com pesos somando 1, o score combinado nunca escapa ao
intervalo dos scores das cabeças – sem explosão, sem ambiguidade (o que
`_validated_weights` impõe por `ValueError`).
- **E3 otimalidade gulosa:** a seleção top-1/top-k retorna candidato(s) de utilidade
máxima – nenhum outro elegível tem utilidade maior. Complexidade `O(n log k)`
documentada como propriedade medida da implementação, não como teorema.

## 3. Teoremas (Lean + Mathlib, `machine_checked` via lake-env R754)

Arquivo: `plugins/lean-nucleo-verificacao/teoremas/roteamento_atencao.lean`

- **T1 `softmax_soma_um`:**  $\sum_i (\exp(f_i) / \sum_j \exp(f_j)) = 1$  (tipo finito não-vazio).
Prova: `Finset.sum_pos` ( $\exp > 0$ ) + `Finset.sum_div` + `div_self`.
- **T2 `combinacao_convexa_limitada`:** pesos não-negativos somando 1 e  $x_i \leq M$ 
implicam  $\sum w_i * x_i \leq M$ . Prova: `Finset.sum_le_sum` + `mul_le_mul_of_nonneg_left`
+ `Finset.sum_mul` + `hsum`.
- **T3 `roteamento_guloso_otimo`:** existe `melhor ∈ candidatos` com utilidade  $\geq$  todas.

```

Prova: ``Finset.exists_max_image``.

- ****T4 `softmax\_invariante\_max` (R756):**** subtrair constante ``c`` não muda os pesos.
Prova: ``Real.exp_sub`` + ``Finset.sum_div`` + ``field_simp`` (fecha sozinho).
- ****T5 `softmax\_mem\_Ioo` (R756):**** cada peso em ``(0,1)`` com ``Nontrivial``.
Prova: ``div_pos`` + ``div_lt_one`` + ``Finset.add_sum_erase`` + ``linarith``.
- ****T6 `combinacao\_convexa\_limitada\_baixo` (R756):**** $m \leq \sum w \cdot x$ (espelho de T2).
- ****T7 `softmax\_peso\_total\_singleton` (R758):**** candidato único recebe peso 1 $\forall (j, j = i)$, complementando T5. Auditoria ``#print axioms``: 7/7 só com ``[propext, Classical.choice, Quot.sound]``, sem ``sorryAx`` (``bancada/axiomas.txt``).
- ****T8 `peso\_cresce\_com\_escore` (R761):**** peso estritamente crescente no próprio escore – justifica a regra de decisão.
- ****T9 `topk\_dominam\_resto` (R761):**** take k de lista pairwise-decrescente domina o resto – o padrão top-k do ranking.
- ****T10 `temperatura\_aguca\_maximo` (R763):**** esfriar aumenta o peso do maximizador único – nitidez sem limites (forma elementar da calibração).
- ****T11 `sem\_overflow\_apos\_max` (R763):**** expoentes ≤ 0 após majorante – ausência de overflow do truque numérico.

4. Aceite (``tests/test_r755_teorema_roteamento.py``)

1. Arquivo ``.lean`` existe com os 11 nomes de teorema e sem ``sorry`/`admit``.
2. ``lean_verify(código, project_dir=demo)`` → ``machine_checked``.
3. ``bancada/axiomas.txt`` contém os 11 nomes e nenhum ``sorryAx``.

5. Contraprova e ablação (R758, ``bancada/``)

- ``contraprova_float.py``: 20k vetores, 0 violações (soma, limites, invariância shift, argmax, singleton, vazio).
- ``bancada_ablacao.py --qw {0.3,0.6,0.9}``: confiança embaralhada/anti/ruidosa, ``media_igual``, ``sem_confianca``, diffs pareados + taxa de vitória.

6. Invioláveis

- Nenhuma alegação de "orquestrador mais eficiente do mundo"; só otimalidade relativa provada.
- ``sorry`/`admit`` bloqueiam; sem projeto lake+Mathlib, máximo ``syntax_verified``.

spec_id: SPEC-935-R754

title: Verificação Lake-Env com Mathlib – machine_checked no Projeto

component: integrations/deepmind/lean4_verifier, integrations/lean_proof_mcp, tests/
test_r754

status: proposed

round_id: R754

test_file: tests/test_r754_lake_env.py

SPEC-935-R754 – Lake-Env com Mathlib

1. Contexto

R444 entrega ``machine_checked`` para Lean núcleo e ``syntax_verified`` com fallback honesto quando Mathlib está ausente no projeto (R753). Com projeto lake + Mathlib pré-compilado via ``lake exe cache get`` (``leanprover/lean4:v4.34.1``, ``mathlib rev v4.34.1``), ``lake env lean`` atingiu exit 0 em 47s para ``simp/omega/ring`` (R754). Falta expor esse caminho no pipeline, sem quebrar o comportamento sem projeto.

2. Requisitos

R754.1 – ``Lean4ProofVerifier.verify_lean_code(code, project_dir=None)``

- ``project_dir=None``: comportamento R444/R753 inalterado (bare ``lean``, fallback Mathlib-ausente).
- ``project_dir`` com ``lakefile.toml`` + ``.lake``: escreve temporário dentro do projeto e executa ``lake env lean <tmp>`` com ``cwd=project_dir``, timeout 300s.
- Exit 0 → ``machine_checked``. ``sorry`/`admit`` continuam bloqueio antes de qualquer execução.
- Erro de prova real → ``compiler_error`` (sem fallback). Ausência de ``lake`/projeto inválido` → fallback para o caminho sem projeto, com nota na proveniência; nunca alegar ``machine_checked``.

R754.2 – MCP ``lean_verify(code, project_dir=None)``

- Repassa ``project_dir``; ``provenance`` inclui ``project_dir`` e ``via: lake-env|bare-lean``.

3. Aceite (``tests/test_r754_lake_env.py``, pula se projeto demo ausente)

1. ``test_machine_checked_mathlib_no_projeto``: ``simp/omega/ring`` com ``import Mathlib.Tactic`` → ``machine_checked``.
2. ``test_sorry_bloqueado_no_projeto``: ``sorry`` → ``is_valid=False``.
3. ``test_prova_errada_falha_no_projeto``: (Uma igualdade falsa, ex. ``2 + 2 = 5`` com ``decide``) → ``compiler_error``, sem fallback para válido.

4. Invioláveis

- Backward compatible: chamada sem ``project_dir`` comporta-se como R753.
- ``sorry`/`admit`/`axiom`` não declarado sempre bloqueiam, com ou sem projeto.
- Sem projeto válido nunca ``machine_checked`` para código com ``import Mathlib``.

spec_id: SPEC-935-R747

title: Núcleo Lean – Verificação Total, Matemática Formal, Tese Auditável e Software Crítico

```
component: agents/catalog, .opencode/skills, integrations/lean_proof_mcp, plugins/lean-
nucleo, tests/test_r747
```

```
status: proposed
```

```
round_id: R747
```

```
test_file: tests/test_r747_nucleo_lean.py
```

```
---
```

```
# SPEC-935-R747 – Núcleo Lean no Core
```

1. Contexto e Motivação

0 Core já possui motores formais verificados:

- R444: `integrations/deepmind/lean4\_verifier.py` + `egraph\_rewriter.py` (status `syntax\_verified`, `machine\_checked` quando há compilador).
- R445: `integrations/deepmind/autoformalizer.py` (informal → Lean 4, Lean 4 → português, validação cruzada).
- R451/R503: sanidade formal e contraprova.

Lacuna: não há **agentes, skills, MCPs e plugins especializados** com contrato auditável para:

1. verificação total fail-closed;
2. matemática formal com Mathlib;
3. tese com provas auditáveis;
4. software crítico;
5. estudo sistemático de teoremas.

Esta SPEC define esse núcleo reutilizando os motores existentes, sem duplicá-los.

Delegação executada: `ecosystem\_run task-99da2230` – Claude falhou por limite de conta, Codex entregou escopo de 15 itens (coordenador, especificador, autoformalizador, verificador, auditor; skills formalizar/auditar; MCP único; plugin único; contrato mínimo; fail-closed `sorry`/`admit`; sem compilador nunca aprovar). Workflow `workflow-4f8179412adf43648ecacb6f98f75421` bloqueou por ausência de agente interno elegível – prova da lacuna.

2. Requisitos e Componentes

R747.1 – Agentes (5 arquivos em `agents/catalog/`)

1. `49\_agente\_lean\_verificacao\_total.md` – coordena o pipeline; impõe fail-closed; nunca declara prova total sem `machine\_checked`.
2. `50\_agente\_lean\_matematica\_formal.md` – formaliza enunciados com Mathlib; fixa imports, versões e táticas permitidas.
3. `51\_agente\_lean\_tese\_auditavel.md` – organiza tese em capítulos com trilha: enunciado PT-BR → código `.lean` → log do verificador → reconciliação.

4. ``52_agente_lean_software_critico.md`` – especifica pré/pós-condições e invariantes; exige ``machine_checked`` para alegar correção.
5. ``53_agente_lean_estudo_teoremas.md`` – estuda teoremas existentes; explica tática por tática; nunca inventa lema Mathlib.

Cada agente: frontmatter YAML (``name``, ``description``, ``tags``, ``type``), seções Missão/Entradas/Saídas/Ativação/Bloqueadores fail-closed/Handoff.

R747.2 – Skills (2 pastas em ``.opencode/skills/``)

1. ``.opencode/skills/lean-verificacao-total/SKILL.md`` – quando acionar, contrato mínimo (enunciado, premissas, código, versão Lean, log), tabela tá-ticauso, bloqueadores (``sorry``, ``admit``, ``axiom`` sem declaração explícita).
2. ``.opencode/skills/lean-tese-provas-auditaveis/SKILL.md`` – estrutura de capítulo auditável, checklist de proveniência Mathlib, modelo de tabela de reconciliação.

Ambas com ``disable-model-invocation: false``, comando explícito, sem execução implícita de scripts de terceiros.

R747.3 – MCP ``integrations/lean_proof_mcp.py``

Funções puras sobre os motores existentes, sem rede:

- ``lean_verify(code: str) -> dict`` – envolve ``Lean4ProofVerifier.verify_lean_code``; ``sorry`` ``/`admit`/`axiom`` `→`is_valid=False``, ``status=incomplete_sorry|axiom_nao_declarado``.
- ``lean_export(theorem_name, statement, tactics, imports) -> dict`` – envolve ``format_theorem``; retorna ``lean_source`` + ``sha256``.
- ``lean_autoformalize(informal_text, domain) -> dict`` – envolve ``AutoFormalizerEngine.informal_to_lean4``.
- ``lean_saturate(expr, rules) -> dict`` – envolve ``EqualitySaturationEngine`` quando disponível; senão ``status=indisponivel``.

Todo retorno inclui ``provenance: {lean_binary, has_compiler, mathlib_version_hint, engine}`` e nunca alega ``machine_checked`` sem compilador real.

R747.4 – Plugin ``plugins/lean-nucleo-verificacao/``

- ``PLUGIN.md`` (manifesto: nome, versão 0.1.0, componentes, licença, uso).
- ``lean-nucleo.json`` (manifesto máquina: agentes, skills, mcp, comandos).
- Reexporta sem copiar lógica dos motores.

3. Critérios de Aceite (SDD/TDD – ``tests/test_r747_nucleo_lean.py``)

1. ``test_spec_r747_registered``: SPEC-935-R747 registrada no ``spec_registry``.
2. ``test_agentes_lean_existem``: 5 arquivos existem com frontmatter e seção Bloqueadores/fail-closed.
3. ``test_skills_lean_existem``: 2 ``SKILL.md`` existem com contrato mínimo e menção a ``sorry`/``

- ```
admit`.
```
4. `test\_mcp\_lean\_verify\_failclosed`: `sorry` → `is\_valid=False`; código limpo com `ring` → `is\_valid=True`, `status` in {syntax\_verified, machine\_checked}.
  5. `test\_mcp\_lean\_export\_hash`: export gera `theorem <nome>` + `sha256` de 64 hex.
  6. `test\_mcp\_lean\_autoformalize`: informal → contém `theorem` e `Mathlib`.
  7. `test\_plugin\_manifesto`: `PLUGIN.md` + `lean-nucleo.json` existem e listam os 5 agentes + 2 skills + MCP.
  8. `test\_sem\_overclaim\_sem\_compilador`: sem binário Lean, nenhum caminho retorna `machine\_checked` como prova total sem ressalva; `has\_compiler=False` → status nunca `machine\_checked` salvo execução real.

## ## 4. Regras Invioláveis

- `sorry`, `admit`, `axiom` não declarado → bloqueio total, sem exceção.
- Sem binário `lean`/`lake` → `syntax\_verified` no máximo; declarar `não validado por compilador`.
- Não declarar Qualis, nota, aceite, superhuman ou prova total sem validação externa explícita.
- Reutilizar motores; não duplicar `lean4\_verifier`, `egraph`, `autoformalizer`.
- Toda mudança relevante registra ciclo em `evolution/cycles.py` após verde.

---

spec\_id: SPEC-935-R767

title: Pipeline de Diagnóstico por Gaps – Auditoria Hermética entre Artefatos

component: scanners/diagnostic\_pipeline, tests/test\_r767\_diagnostic\_pipeline

status: proposed

round\_id: R767

test\_file: tests/test\_r767\_diagnostic\_pipeline.py

---

# SPEC-935-R767 – Pipeline de Diagnóstico por Gaps

## ## 1. Motivação

Revisões por gaps (R766) hoje são manuais. Este módulo automatiza a parte mecânica de forma hermética: sem rede, subprocesso ou LLM. Não substitui compilação Lean, execução de testes ou validação externa; encontra derivas de sincronia (nomes, cobertura, tokens proibidos) antes delas virarem falha.

## ## 2. Requisitos

### R767.1 – `DiagnosticPipeline.varrer(plano: dict) -> dict`

Plano com duas checagens:

1. `{"tipo": "nomes", "id", "arquivo", "regex", "esperados": [...], "proibidos": [...]}` – extrai nomes via regex com 1 grupo; acusa

faltantes, excedentes e ocorrências de padrões proibidos.

2. `{"tipo": "cobertura", "id", "arquivo", "regex", "referencia": {"arquivo", "regex"}}` – todo nome de A deve ocorrer em B.

Retorno: `{spec_id, gerador, total_checks, passaram, falharam, gaps: [{id, motivo}], rotulo: candidato_a_inspecao, exige_validacao_externa: True}`. Arquivo ausente ou ilegível vira gap (nunca exceção); plano malformado levanta `ValueError` (fail-closed).

## 3. Aceite (`tests/test_r767_diagnostic_pipeline.py`, hermético, `tmp_path`)

1. Plano consistente → 0 gaps.
2. Nome esperado ausente → gap `faltante`.
3. Token proibido presente → gap `proibido`.
4. Nome sem cobertura → gap `cobertura`.
5. Plano malformado → `ValueError`.
6. Arquivo ausente → gap (sem exceção).

## 4. Invioláveis

- Sem rede, subprocesso, LLM ou import de terceiros além da `stdlib`.
- Rótulo sempre `candidato_a_inspecao`; nada é declarado verificado.